

Neural Network Architectures

This guide is written for placement interviews, AI engineer roles, research internships, and project preparation. Each architecture is explained from basic intuition to practical implementation and interview depth.

Feedforward Neural Network

1. Overview

A Feedforward Neural Network, also called a Multilayer Perceptron (MLP), is the basic neural network architecture where information flows in one direction: input layer to hidden layers to output layer. There are no cycles, memory states, or recurrence.

It is useful because it can approximate complex non-linear functions. In real systems, MLPs are used for tabular prediction, feature fusion, recommendation ranking, classification heads on top of embeddings, and simple regression/classification tasks.

2. Intuition

Think of an MLP as a chain of decision layers. The first layer learns simple patterns, later layers combine them into more useful patterns, and the final layer converts them into a prediction.

Example: for house price prediction, one neuron may learn that area matters, another may learn that location matters, and later neurons combine area, location, number of rooms, and age into a final price estimate.

3. Prerequisites

- Linear algebra: vectors, matrices, dot products
- Calculus: derivatives, chain rule
- Probability: softmax, cross-entropy
- ML basics: supervised learning, train/validation/test split
- Deep learning: activation functions, backpropagation, gradient descent
- PyTorch basics: tensors, modules, optimizers

4. Core Concepts

Subtopic	Meaning	Why it matters	Simple example	Interview angle
Neuron	Computes weighted sum plus bias and activation	Basic computation unit	$y = \text{ReLU}(w \cdot x + b)$	Difference between neuron and perceptron
Layer	Group of neurons operating together	Learns multiple features	Dense layer with 128 neurons	Why deeper networks learn hierarchy

Subtopic	Meaning	Why it matters	Simple example	Interview angle
Activation	Non-linear function	Without it, many layers collapse to one linear model	ReLU, sigmoid, tanh	Why ReLU is common
Loss	Error measure	Guides learning	Cross-entropy for classification	Choose correct loss for task
Backpropagation	Gradient computation using chain rule	Updates weights efficiently	Compute gradients layer by layer	Vanishing gradients and chain rule
Regularization	Prevents overfitting	Improves generalization	Dropout, weight decay	Dropout train vs eval behavior

5. Algorithm / Working Process

1. Input: feature vector $\mathbf{x}$.
2. First hidden layer computes $\mathbf{h1} = \text{activation}(\mathbf{W1x} + \mathbf{b1})$.
3. Later hidden layers repeat the same pattern.
4. Output layer produces logits or continuous values.
5. Training: compute loss, run backpropagation, update weights with an optimizer.
6. Inference: run only the forward pass and convert output to class/probability/value.

6. Mathematical Foundation

For layer l :

$$\begin{aligned} z[l] &= W[l]a[l-1] + b[l] \\ a[l] &= g(z[l]) \end{aligned}$$

For binary classification:

$$\begin{aligned} p &= \text{sigmoid}(z) = 1 / (1 + e^{-z}) \\ L &= -[y \log(p) + (1-y) \log(1-p)] \end{aligned}$$

For multi-class classification:

$$\begin{aligned} \text{softmax}(z_i) &= e^{(z_i)} / \sum_j e^{(z_j)} \\ L &= -\sum_i y_i \log(p_i) \end{aligned}$$

Optimization:

```
theta = theta - learning_rate * gradient(L, theta)
```

7. Practical Implementation

```
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

X = torch.randn(1000, 20)
y = (X[:, 0] + 0.5 * X[:, 1] > 0).long()
loader = DataLoader(TensorDataset(X, y), batch_size=32, shuffle=True)

class MLP(nn.Module):
    def __init__(self, in_features=20, hidden=64, classes=2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_features, hidden),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(hidden, hidden),
            nn.ReLU(),
            nn.Linear(hidden, classes),
        )

    def forward(self, x):
        return self.net(x)

model = MLP()
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(5):
    for xb, yb in loader:
        optimizer.zero_grad()
        loss = loss_fn(model(xb), yb)
        loss.backward()
        optimizer.step()

    with torch.no_grad():
        preds = model(X).argmax(dim=1)
        accuracy = (preds == y).float().mean().item()
    print(f"accuracy={accuracy:.3f}")
```

8. Code Explanation

`TensorDataset` and `DataLoader` create mini-batches. `nn.Sequential` defines the layer stack. `CrossEntropyLoss` expects raw logits, not softmax probabilities. `optimizer.zero_grad()` clears old gradients, `loss.backward()` computes gradients, and `optimizer.step()` updates weights.

9. Training / Evaluation

Use normalization for numerical features. Split data into train, validation, and test sets. Use accuracy, precision, recall, F1, ROC-AUC, MAE, MSE, or RMSE depending on task. Watch for overfitting by comparing train and validation loss. Tune hidden size, depth, dropout, learning rate, batch size, and weight decay.

10. Complexity and Cost

For one dense layer, computation is $O(\text{input_dim} * \text{output_dim})$. MLPs are cheap compared with CNNs, Transformers, and diffusion models. Memory is mostly parameters plus activations stored for backpropagation.

11. Common Use Cases

- Tabular classification/regression
- Recommendation ranking heads
- Fraud detection
- Feature fusion from multiple models
- Classification layers after CNN/Transformer encoders

12. Common Mistakes

- Applying softmax before `CrossEntropyLoss`
- Not scaling numerical features
- Using accuracy on imbalanced data
- Too many layers for small tabular data
- Forgetting `model.eval()` during evaluation
- Data leakage during preprocessing

13. Edge Cases / Limitations

MLPs do not naturally exploit spatial structure, sequence order, or graph structure. They can overfit small datasets and may perform worse than tree models on tabular data.

14. Variations

Variation	What changes	When to use	Placement importance
Shallow MLP	One hidden layer	Simple tabular tasks	High
Deep MLP	Many hidden layers	Complex feature interactions	High
Residual MLP	Skip connections	Very deep dense networks	Medium
MLP-Mixer	MLPs over image patches	Vision alternative to CNN/ViT	Medium

15. Related Topics

CNNs add spatial inductive bias. RNNs add sequence memory. Transformers use attention instead of fixed dense-only processing. Logistic regression is an MLP without hidden non-linear layers.

16. Interview Questions

1. What is a feedforward neural network?
A neural network where information flows from input to output without cycles.
2. Why are activation functions needed?
They add non-linearity; otherwise stacked linear layers become one linear layer.
3. Why is ReLU popular?
It is simple, fast, and reduces vanishing gradient issues for positive inputs.
4. What is backpropagation?
Efficient gradient computation using the chain rule.
5. What loss is used for multi-class classification?
Cross-entropy loss with softmax internally.
6. What causes overfitting in MLPs?
Too many parameters, small data, poor regularization, leakage.
7. How do dropout and weight decay differ?
Dropout randomly disables activations; weight decay penalizes large weights.
8. Why normalize inputs?
It stabilizes and speeds up optimization.
9. What is the difference between logits and probabilities?
Logits are raw scores; probabilities are normalized outputs.
10. When should you avoid MLPs?
When data has strong image, sequence, or graph structure better handled by specialized architectures.

17. Practice Tasks

- Build an MLP for Iris or Breast Cancer classification.
- Compare MLP and Random Forest on a tabular dataset.
- Experiment with dropout values.
- Debug a model where softmax is incorrectly applied before cross-entropy.
- Add early stopping using validation loss.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Loan Default Predictor	Predicts loan repayment risk	PyTorch, pandas, sklearn	LendingClub/Kaggle	Strong tabular ML project
Student Placement Predictor	Predicts placement chance	PyTorch, FastAPI	Campus placement dataset	Direct placement relevance
Fraud Score API	Serves fraud probability	PyTorch, FastAPI, Docker	Credit card fraud dataset	Shows deployment skill

19. Quick Revision

- Key idea: stack linear layers and activations to learn non-linear mappings.
- Main formula: $a[l] = g(W[l]a[l-1] + b[l])$.
- When to use: tabular data, feature fusion, simple classification/regression.
- Important metrics: accuracy, F1, ROC-AUC, MAE, RMSE.
- Common traps: no scaling, softmax before cross-entropy, overfitting.
- Interview one-liner: an MLP is the basic universal function approximator in deep learning.

20. Final Cheat Sheet

Item	Answer
Definition	Dense feedforward neural network
Input/output	Feature vector to class/value
Main steps	Linear transformation, activation, loss, backprop
Key hyperparameters	Layers, hidden units, LR, dropout, batch size
Metrics	Accuracy, F1, ROC-AUC, MSE
Pros	Simple, flexible, fast
Cons	Weak for images/sequences/graphs without feature engineering
Best use cases	Tabular prediction and neural classifier heads

CNN

1. Overview

A Convolutional Neural Network is designed for grid-like data, especially images. It uses convolution filters to detect local patterns such as edges, textures, object parts, and full objects.

CNNs are used in image classification, object detection, segmentation, OCR, medical imaging, satellite analysis, autonomous driving perception, and video understanding.

2. Intuition

Instead of connecting every pixel to every neuron, CNNs scan small filters across an image. A filter may detect vertical edges wherever they appear. Deeper filters combine edges into shapes, then objects.

3. Prerequisites

- Image representation: channels, height, width
- Matrix operations and convolution
- Backpropagation
- Activation functions
- Pooling and padding
- PyTorch tensor shape convention: **NCHW**

4. Core Concepts

Subtopic	Meaning	Why it matters	Simple example	Interview angle
Convolution	Sliding filter dot product	Shares weights and captures local patterns	3x3 edge detector	Why fewer parameters than MLP
Kernel/filter	Learnable local pattern detector	Learns visual features	Sobel-like filter	Kernel size tradeoff
Stride	Step size of filter movement	Controls output size	Stride 2 downsamples	Effect on feature map size
Padding	Border extension	Preserves spatial size	Same padding	Formula for output size
Pooling	Downsampling operation	Adds translation tolerance	MaxPool 2x2	Max vs average pooling
Channels	Feature maps	Multiple learned patterns	64 filters produce 64 channels	Input/output channel relation

5. Algorithm / Working Process

Input image tensor has shape `[batch, channels, height, width]`. Convolution layers compute feature maps. Activations add non-linearity. Pooling or strided convolution reduces spatial resolution. A classifier head maps final features to class logits. Training uses backpropagation with image augmentations. Inference applies the same forward pass without augmentation except deterministic preprocessing.

6. Mathematical Foundation

2D convolution:

$$Y(i,j,k) = \sum_c \sum_u \sum_v X(i+u,j+v,c) * W(u,v,c,k) + b_k$$

Output size:

$$H_{out} = \text{floor}((H + 2P - K) / S) + 1$$

$$W_{out} = \text{floor}((W + 2P - K) / S) + 1$$

Classification loss is usually cross-entropy.

7. Practical Implementation

```
import torch
from torch import nn
```

```

class SmallCNN(nn.Module):
    def __init__(self, classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.AdaptiveAvgPool2d((1, 1)),
            nn.Flatten(),
            nn.Linear(64, classes),
        )

    def forward(self, x):
        return self.classifier(self.features(x))

model = SmallCNN()
x = torch.randn(8, 3, 32, 32)
logits = model(x)
print(logits.shape)

```

8. Code Explanation

`Conv2d` learns image filters. `BatchNorm2d` stabilizes training. `MaxPool2d` reduces resolution.

`AdaptiveAvgPool2d((1,1))` allows the classifier to work with different image sizes. The model outputs raw class logits.

9. Training / Evaluation

Use train/validation/test split by image identity, not by augmented copies. Apply resize, normalization, random crop, flip, color jitter where appropriate. Metrics include accuracy, top-k accuracy, F1, mean IoU for segmentation, and mAP for detection. Tune learning rate, augmentation, batch size, depth, and weight decay.

10. Complexity and Cost

Convolution cost is approximately $O(H * W * C_{in} * C_{out} * K^2)$. CNNs can be efficient because weights are shared. Larger images and many channels increase memory and GPU cost.

11. Common Use Cases

- Image classification
- Face recognition
- Medical image diagnosis
- OCR feature extraction

- Defect detection in manufacturing
- Object detection and segmentation backbones

12. Common Mistakes

- Wrong channel order: **NHWC** instead of **NCHW** in PyTorch
- Not normalizing images
- Leakage from same patient/object across splits
- Too little augmentation
- Flattening too early and creating huge dense layers
- Ignoring class imbalance

13. Edge Cases / Limitations

CNNs have locality bias and may struggle with long-range relationships unless deep or combined with attention. They can be sensitive to domain shift, adversarial perturbations, and poor lighting/camera conditions.

14. Variations

Variation	What changes	When to use	Importance
1D CNN	Convolve over time/text	Signals, text classification	High
3D CNN	Convolve over volume/time	CT scans, video	Medium
Depthwise separable CNN	Splits spatial and channel mixing	Mobile models	High
Dilated CNN	Adds gaps in kernels	Segmentation, larger receptive field	Medium

15. Related Topics

CNN vs MLP: CNN shares weights and preserves spatial structure. CNN vs ViT: CNN has local inductive bias, ViT uses global attention over patches. CNNs are often used inside ResNet and U-Net.

16. Interview Questions

1. Why are CNNs better than MLPs for images?
They use local connectivity, weight sharing, and spatial hierarchy.
2. What is a kernel?
A learnable filter that detects local patterns.
3. What does padding do?
Controls border handling and output size.
4. How does stride affect output?
Larger stride reduces spatial resolution.
5. What is pooling?
Downsampling that summarizes local regions.

6. What is receptive field?
The region of input influencing a feature.
7. Why use batch normalization?
It stabilizes training and often allows faster convergence.
8. What is depthwise separable convolution?
A cheaper convolution used in mobile models.
9. Why avoid large dense layers after conv layers?
They add many parameters and overfit.
10. What is translation equivariance?
A shifted input creates a shifted feature map.

17. Practice Tasks

- Train a CNN on CIFAR-10.
- Calculate output size for different kernel/stride/padding values.
- Compare max pooling and strided convolution.
- Debug an image pipeline with wrong normalization.
- Visualize first-layer filters.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Plant Disease Classifier	Classifies leaf disease	PyTorch, torchvision	PlantVillage	CV project with real impact
Defect Detection	Finds defective products	PyTorch, OpenCV	MVTec AD	Industrial AI relevance
Traffic Sign Classifier	Recognizes road signs	PyTorch	GTSRB	Good autonomous systems project

19. Quick Revision

- Key idea: learn local image filters with shared weights.
- Main formula: output size $\text{floor}((H+2P-K)/S)+1$.
- When to use: image/grid data.
- Important metrics: accuracy, mAP, IoU.
- Common traps: wrong shape, no normalization, leakage.
- Interview one-liner: CNNs exploit spatial locality and weight sharing.

20. Final Cheat Sheet

Item	Answer
Definition	Neural network using convolution filters
Input/output	Image tensor to class/map/features
Main steps	Conv, activation, pooling, head

Item	Answer
Key hyperparameters	Kernel, stride, padding, channels, depth
Metrics	Accuracy, F1, IoU, mAP
Pros	Efficient, strong image bias
Cons	Weaker global context than attention
Best use cases	Images, videos, spatial signals

RNN

1. Overview

A Recurrent Neural Network processes sequences by maintaining a hidden state across time steps. It is designed for ordered data such as text, speech, time series, and sensor streams.

RNNs were central in early NLP and speech systems. Today, Transformers dominate many sequence tasks, but RNNs remain useful for small, streaming, low-latency, and embedded sequence models.

2. Intuition

An RNN reads one token at a time while carrying a memory. If the sentence is "I grew up in India, so I speak ...", the hidden state should help predict "Hindi" or another language because previous words matter.

3. Prerequisites

- Sequence data and time steps
- Hidden states
- Matrix multiplication
- Backpropagation through time
- Gradient vanishing/exploding
- Tokenization or time-series preprocessing

4. Core Concepts

Subtopic	Meaning	Why it matters	Simple example	Interview angle
Hidden state	Memory vector	Carries past information	Previous words in sentence	Why RNN handles variable length
Recurrence	Same cell reused over time	Parameter sharing across positions	Same update for each word	Unrolling RNN
BPTT	Backprop through time	Trains recurrent weights	Gradients flow across steps	Vanishing gradients
Many-to-one	Sequence to label	Classification	Sentiment analysis	Which output to use

Subtopic	Meaning	Why it matters	Simple example	Interview angle
Many-to-many	Sequence to sequence	Tagging or translation	POS tagging	Alignment issues

5. Algorithm / Working Process

At each time step, the RNN receives input x_t and previous hidden state h_{t-1} . It computes new hidden state h_t . For classification, the final hidden state is passed to a dense layer. For sequence labeling, every hidden state may produce an output.

6. Mathematical Foundation

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$$

$$y_t = W_{hy} h_t + b_y$$

Training uses backpropagation through time. The main problem is repeated multiplication by recurrent weights, which can shrink or explode gradients:

$$dL/dh_t \text{ contains product of many } W_{hh} \text{ terms}$$

7. Practical Implementation

```
import torch
from torch import nn

class RNNClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim=64, hidden=128, classes=2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.RNN(embed_dim, hidden, batch_first=True)
        self.fc = nn.Linear(hidden, classes)

    def forward(self, tokens):
        x = self.embedding(tokens)
        output, hidden = self.rnn(x)
        last_hidden = hidden[-1]
        return self.fc(last_hidden)

model = RNNClassifier(vocab_size=5000)
tokens = torch.randint(0, 5000, (16, 30))
logits = model(tokens)
print(logits.shape)
```

8. Code Explanation

Embedding converts token IDs into vectors. `nn.RNN` processes the sequence. `hidden[-1]` is the final hidden state from the last recurrent layer. The final linear layer maps sequence representation to class logits.

9. Training / Evaluation

Pad variable-length sequences and use packed sequences if needed. Use cross-entropy for classification or sequence labeling. Monitor gradient norms. Use validation metrics such as accuracy, F1, perplexity, MAE, or RMSE depending on the task.

10. Complexity and Cost

RNN time complexity is $O(T * \text{hidden}^2)$ approximately, where T is sequence length. It is sequential across time, so it cannot parallelize as well as Transformers during training.

11. Common Use Cases

- Time-series forecasting
- Sentiment classification
- Speech sequence modeling
- Character-level language modeling
- Embedded streaming models

12. Common Mistakes

- Ignoring padding masks
- Using final hidden state for padded tokens
- Forgetting gradient clipping
- Expecting vanilla RNNs to learn long dependencies
- Mixing batch-first and sequence-first tensor formats

13. Edge Cases / Limitations

Vanilla RNNs struggle with long-term dependencies due to vanishing gradients. Training is slow for long sequences because computation is sequential.

14. Variations

Variation	What changes	When to use	Importance
Bidirectional RNN	Reads forward and backward	Full sequence available	High
Stacked RNN	Multiple recurrent layers	More capacity	Medium
LSTM	Adds gates and cell state	Long dependencies	High
GRU	Simpler gated RNN	Efficient sequence model	High

15. Related Topics

RNN vs LSTM: LSTM adds gates to improve memory. RNN vs Transformer: Transformer parallelizes and uses attention. RNN vs CNN for sequences: CNN sees local windows; RNN maintains state over time.

16. Interview Questions

1. What makes RNNs suitable for sequences?
They maintain hidden state across time.
2. What is BPTT?
Backpropagation applied to the unrolled sequence.
3. Why do RNNs suffer from vanishing gradients?
Gradients are repeatedly multiplied through time.
4. What is hidden state?
A vector summary of previous inputs.
5. What is many-to-one modeling?
Mapping a sequence to one output.
6. What is many-to-many modeling?
Producing output at multiple time steps.
7. Why use gradient clipping?
To prevent exploding gradients.
8. Can RNNs process variable-length sequences?
Yes, with padding/masking or packed sequences.
9. Why are Transformers faster to train?
They parallelize across tokens.
10. When are RNNs still useful?
Streaming, small-data, low-latency sequence tasks.

17. Practice Tasks

- Build a character-level name classifier.
- Train an RNN for stock movement direction.
- Compare RNN, LSTM, and GRU on the same dataset.
- Debug padding-related metric errors.
- Add gradient clipping and compare stability.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Sentiment RNN	Classifies reviews	PyTorch	IMDb	NLP fundamentals
Sensor Fault Predictor	Predicts machine failure	PyTorch, pandas	NASA turbofan	Time-series ML
Character Name Generator	Generates names	PyTorch	Name lists	Sequence generation

19. Quick Revision

- Key idea: hidden state carries sequence memory.
- Main formula: $h_t = \tanh(Wx_t + Uh_{t-1} + b)$.
- When to use: ordered sequence data.
- Important metrics: F1, perplexity, MAE/RMSE.

- Common traps: padding, vanishing gradients.
- Interview one-liner: RNNs reuse the same cell across time to process sequences.

20. Final Cheat Sheet

Item	Answer
Definition	Recurrent sequence neural network
Input/output	Sequence to label/sequence
Main steps	Embed, recurrent update, output head
Key hyperparameters	Hidden size, layers, sequence length, LR
Metrics	Accuracy, F1, perplexity, RMSE
Pros	Handles variable-length sequences
Cons	Slow and weak on long dependencies
Best use cases	Streaming sequence models

LSTM

1. Overview

Long Short-Term Memory is a gated RNN architecture designed to remember information over longer sequences. It adds a cell state and gates that control what to forget, store, and output.

LSTMs are used in time-series forecasting, speech recognition, handwriting recognition, anomaly detection, and older NLP systems.

2. Intuition

An LSTM is like a notebook with three controls: erase irrelevant notes, write useful new notes, and decide what part of the notes to reveal for the current prediction.

3. Prerequisites

- RNNs and hidden states
- Sigmoid and tanh
- Element-wise multiplication
- Sequence padding/masking
- Backpropagation through time

4. Core Concepts

Subtopic	Meaning	Why it matters	Simple example	Interview angle
Cell state	Long-term memory path	Helps preserve gradients	Topic of paragraph	Difference from hidden state
Forget gate	Decides what to remove	Removes irrelevant past	Forget old subject	Why sigmoid
Input gate	Decides what to write	Stores new useful info	Store negation word	Gate equations
Output gate	Decides what to expose	Controls hidden output	Reveal sentiment info	Hidden vs cell state
Gates	Learnable filters	Dynamic memory control	Values between 0 and 1	LSTM vs vanilla RNN

5. Algorithm / Working Process

For each time step, LSTM combines current input and previous hidden state. It computes forget, input, candidate, and output values. The cell state is updated using forget and input gates. The hidden state is computed from the updated cell state and output gate.

6. Mathematical Foundation

```

f_t = sigmoid(W_f [h_{t-1}, x_t] + b_f)
i_t = sigmoid(W_i [h_{t-1}, x_t] + b_i)
g_t = tanh(W_g [h_{t-1}, x_t] + b_g)
c_t = f_t * c_{t-1} + i_t * g_t
o_t = sigmoid(W_o [h_{t-1}, x_t] + b_o)
h_t = o_t * tanh(c_t)

```

The additive cell update helps gradients flow better than vanilla RNN recurrence.

7. Practical Implementation

```

import torch
from torch import nn

class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim=64, hidden=128, classes=2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden, batch_first=True,
                             bidirectional=True)
        self.fc = nn.Linear(hidden * 2, classes)

    def forward(self, tokens):
        x = self.embedding(tokens)

```

```
        output, (hidden, cell) = self.lstm(x)
        final = torch.cat([hidden[-2], hidden[-1]], dim=1)
        return self.fc(final)

model = LSTMClassifier(8000)
tokens = torch.randint(0, 8000, (16, 40))
print(model(tokens).shape)
```

8. Code Explanation

`nn.LSTM` returns outputs for all time steps plus final hidden and cell states. Because the model is bidirectional, the last two hidden states correspond to forward and backward directions. Concatenating them gives a stronger sequence representation.

9. Training / Evaluation

Use padding masks or packed sequences for variable length data. Apply gradient clipping. Tune hidden size, number of layers, dropout, and learning rate. Evaluate using F1 for classification, RMSE/MAE for forecasting, or word error rate for speech.

10. Complexity and Cost

LSTM has about four times the recurrent computations of vanilla RNN because it has four gate-related transformations. It is still sequential across time.

11. Common Use Cases

- Time-series forecasting
- Sentiment analysis
- Named entity recognition
- Speech and handwriting recognition
- Sequence anomaly detection

12. Common Mistakes

- Confusing cell state and hidden state
- Ignoring bidirectional output shape
- Using LSTM for very long text where Transformer is better
- Not masking padded tokens
- Forgetting gradient clipping

13. Edge Cases / Limitations

LSTMs handle longer dependencies than RNNs but still struggle with very long contexts. Training is slower than attention-based parallel models.

14. Variations

Variation	What changes	When to use	Importance
BiLSTM	Uses future and past context	Tagging/classification	High
Stacked LSTM	More layers	More expressive sequences	Medium
ConvLSTM	Convolution inside gates	Video/weather grids	Medium
Peephole LSTM	Gates see cell state	Fine timing tasks	Low

15. Related Topics

LSTM vs GRU: GRU is simpler and often faster. LSTM vs Transformer: LSTM has recurrence; Transformer uses attention. LSTM is a solution to vanilla RNN gradient issues.

16. Interview Questions

1. Why was LSTM introduced?
To reduce vanishing gradient problems in vanilla RNNs.
2. What are the three main gates?
Forget, input, and output gates.
3. What is the cell state?
A long-term memory vector updated additively.
4. Why does LSTM remember longer dependencies?
Gates control memory and the cell path supports gradient flow.
5. What is a BiLSTM?
An LSTM that reads sequences forward and backward.
6. LSTM vs GRU?
LSTM has separate cell state and more gates; GRU is simpler.
7. Why use sigmoid in gates?
It outputs values between 0 and 1 for soft selection.
8. What does the forget gate do?
It scales old cell state information.
9. Is LSTM parallelizable across time?
Not fully, because each step depends on the previous state.
10. When would you choose LSTM today?
Time series, streaming, or smaller sequence tasks.

17. Practice Tasks

- Train BiLSTM sentiment classifier.
- Forecast temperature using LSTM.
- Compare hidden sizes on validation loss.
- Debug shape errors in bidirectional LSTM.
- Add packed padded sequences.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Energy Forecasting	Predicts electricity demand	PyTorch, pandas	UCI household power	Time-series project
BiLSTM NER	Extracts entities	PyTorch	CoNLL-2003	NLP sequence labeling
Anomaly Detector	Flags abnormal sensor streams	PyTorch	SWaT/NASA	Industrial ML

19. Quick Revision

- Key idea: gated memory with cell state.
- Main formula: $c_t = f_t * c_{t-1} + i_t * g_t$.
- When to use: sequences with medium/long dependencies.
- Important metrics: F1, RMSE, WER.
- Common traps: hidden/cell confusion, padding.
- Interview one-liner: LSTM is an RNN with gates that control long-term memory.

20. Final Cheat Sheet

Item	Answer
Definition	Gated recurrent network
Input/output	Sequence to hidden states/predictions
Main steps	Forget, input, cell update, output
Key hyperparameters	Hidden size, layers, dropout, LR
Metrics	F1, RMSE, WER
Pros	Better long-term memory than RNN
Cons	Slower and heavier than GRU
Best use cases	Forecasting and sequence labeling

GRU

1. Overview

Gated Recurrent Unit is a simpler gated RNN that merges some LSTM ideas into fewer gates. It uses reset and update gates and does not maintain a separate cell state.

GRUs are used for time series, small NLP systems, speech, and real-time sequence processing where LSTM may be heavier than needed.

2. Intuition

A GRU asks two questions at each time step: how much old memory should be kept, and how much past information should be ignored while forming new memory.

3. Prerequisites

- RNN basics
- LSTM intuition
- Sigmoid/tanh
- Hidden states
- Sequence batching and masking

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Update gate	Controls old vs new state	Similar to memory retention	Keep old sentiment context	Compare with LSTM forget/input
Reset gate	Controls past usage	Helps ignore irrelevant history	Reset after sentence boundary	Gate interpretation
Candidate state	Proposed new memory	Adds new sequence info	Current word meaning	Formula
No cell state	Hidden state is memory	Simpler than LSTM	One state vector	Speed vs expressiveness

5. Algorithm / Working Process

At each time step, compute reset and update gates. Use reset gate to build candidate memory. Use update gate to interpolate between old hidden state and candidate state.

6. Mathematical Foundation

```

z_t = sigmoid(W_z x_t + U_z h_{t-1})
r_t = sigmoid(W_r x_t + U_r h_{t-1})
h~_t = tanh(W_h x_t + U_h (r_t * h_{t-1}))
h_t = (1 - z_t) * h_{t-1} + z_t * h~_t

```

Some libraries use the opposite convention for z_t , but the idea is interpolation between old and new memory.

7. Practical Implementation

```

import torch
from torch import nn

class GRUForecaster(nn.Module):

```

```
def __init__(self, features=4, hidden=64, horizon=1):
    super().__init__()
    self.gru = nn.GRU(features, hidden, batch_first=True)
    self.head = nn.Linear(hidden, horizon)

def forward(self, x):
    output, hidden = self.gru(x)
    return self.head(hidden[-1])

model = GRUForecaster()
x = torch.randn(32, 24, 4)
prediction = model(x)
print(prediction.shape)
```

8. Code Explanation

The input shape is `[batch, time, features]`. `nn.GRU` processes the time dimension. `hidden[-1]` is the final sequence summary. The linear head predicts the next value or horizon.

9. Training / Evaluation

For forecasting, create sliding windows from time-series data and split by time, not random rows. Use MAE, RMSE, MAPE, or pinball loss. For classification, use cross-entropy and F1. Tune sequence length, hidden size, learning rate, and batch size.

10. Complexity and Cost

GRU is cheaper than LSTM because it uses fewer gates. Complexity is still sequential in `T`, roughly $O(T * \text{hidden}^2)$.

11. Common Use Cases

- Demand forecasting
- Sensor prediction
- Text classification
- Speech sequence modeling
- Real-time sequence systems

12. Common Mistakes

- Random splitting time series
- Leakage from future values
- Not scaling continuous features
- Using too long a sequence without need
- Confusing GRU update gate with LSTM output gate

13. Edge Cases / Limitations

GRUs may be less expressive than LSTMs on tasks requiring precise long memory. Like all RNNs, training is not fully parallel across time.

14. Variations

Variation	What changes	When to use	Importance
BiGRU	Reads both directions	Text classification/tagging	High
Stacked GRU	Multiple GRU layers	More capacity	Medium
Attention GRU	Adds attention over states	Better long sequence summaries	Medium

15. Related Topics

GRU vs LSTM: GRU is simpler and faster; LSTM has richer memory control. GRU vs Transformer: GRU is recurrent and streaming-friendly; Transformer is parallel and context-rich.

16. Interview Questions

1. What is a GRU?
A gated RNN with update and reset gates.
2. How is GRU different from LSTM?
GRU has fewer gates and no separate cell state.
3. What does update gate control?
How much new candidate state replaces old hidden state.
4. What does reset gate control?
How much past state influences candidate memory.
5. Why can GRU train faster than LSTM?
It has fewer parameters and computations.
6. Does GRU solve vanishing gradients completely?
No, but it reduces the issue compared with vanilla RNNs.
7. When prefer GRU over LSTM?
When speed and simplicity matter and performance is similar.
8. What tensor shape does PyTorch GRU expect with `batch_first=True`?
`[batch, time, features]`.
9. Can GRU be bidirectional?
Yes, if future context is available.
10. Is GRU good for streaming?
Yes, because it updates state step by step.

17. Practice Tasks

- Forecast daily temperature with GRU.
- Compare GRU vs LSTM validation time and accuracy.
- Build BiGRU text classifier.
- Debug future leakage in forecasting.
- Add teacher forcing for sequence prediction.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Sales Forecast GRU	Predicts sales	PyTorch, pandas	Rossmann/Kaggle	Business forecasting
Wearable Activity Model	Classifies activity	PyTorch	UCI HAR	Sensor ML
News Sentiment GRU	Classifies finance headlines	PyTorch	Financial phrasebank	Finance NLP

19. Quick Revision

- Key idea: simpler gated sequence model.
- Main formula: $h_t = (1 - z_t)h_{t-1} + z_t \tilde{h}_t$.
- When to use: efficient sequence modeling.
- Important metrics: F1, MAE, RMSE.
- Common traps: time leakage, padding.
- Interview one-liner: GRU is a lighter LSTM-style recurrent model.

20. Final Cheat Sheet

Item	Answer
Definition	Gated recurrent unit
Input/output	Sequence to hidden/prediction
Main steps	Reset gate, update gate, candidate state
Key hyperparameters	Hidden size, layers, sequence length
Metrics	F1, MAE, RMSE
Pros	Faster than LSTM
Cons	Less expressive memory than LSTM
Best use cases	Forecasting and real-time sequence tasks

Encoder-decoder

1. Overview

An encoder-decoder architecture maps an input sequence or structure into an intermediate representation, then decodes that representation into an output sequence or structure. It is central to translation, summarization, speech recognition, image captioning, and many generative models.

2. Intuition

The encoder reads and understands the input. The decoder uses that understanding to generate the output. For translation, the encoder reads English; the decoder writes Hindi.

3. Prerequisites

- Sequence models
- Embeddings
- Teacher forcing
- Cross-entropy loss
- Autoregressive generation
- Attention, for modern encoder-decoder models

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Encoder	Converts input to representation	Captures source information	Sentence embedding	Bottleneck problem
Decoder	Generates output	Produces target tokens	Translation output	Autoregressive decoding
Context vector	Encoded summary	Bridges encoder and decoder	Final RNN state	Why attention helps
Teacher forcing	Feed true previous token during training	Stabilizes learning	Use real previous word	Exposure bias
Beam search	Keeps multiple candidate outputs	Improves generation	Top 5 translations	Greedy vs beam

5. Algorithm / Working Process

Encoder receives input tokens and produces hidden states. Decoder starts with a start token. At each step, it predicts the next token using previous target token and encoder representation. Training uses target sequence shifted right. Inference generates one token at a time until end token or max length.

6. Mathematical Foundation

Conditional generation:

$$P(y \mid x) = \prod_t P(y_t \mid y_{<t}, x)$$

Training loss:

$$L = -\sum_t \log P(y_t \mid y_{<t}, x)$$

Without attention, decoder depends heavily on final encoder state. With attention, it uses all encoder states.

7. Practical Implementation

```
import torch
from torch import nn

class TinySeq2Seq(nn.Module):
    def __init__(self, vocab, emb=64, hidden=128):
        super().__init__()
        self.embed = nn.Embedding(vocab, emb)
        self.encoder = nn.GRU(emb, hidden, batch_first=True)
        self.decoder = nn.GRU(emb, hidden, batch_first=True)
        self.out = nn.Linear(hidden, vocab)

    def forward(self, src, tgt_in):
        _, h = self.encoder(self.embed(src))
        dec_out, _ = self.decoder(self.embed(tgt_in), h)
        return self.out(dec_out)

model = TinySeq2Seq(vocab=1000)
src = torch.randint(0, 1000, (4, 12))
tgt_in = torch.randint(0, 1000, (4, 10))
logits = model(src, tgt_in)
print(logits.shape)
```

8. Code Explanation

The encoder reads `src` and returns final hidden state `h`. The decoder receives shifted target input `tgt_in` and starts from `h`. The output layer predicts vocabulary logits for each target time step.

9. Training / Evaluation

Use paired source-target data. Tokenize both sides. During training, use teacher forcing with shifted target tokens. Evaluate with BLEU, ROUGE, exact match, token accuracy, or task-specific metrics. Watch train-inference mismatch because inference uses generated tokens, not ground truth.

10. Complexity and Cost

RNN encoder-decoders are sequential in source and target length. Transformer encoder-decoders are more parallel during training but attention cost is quadratic in sequence length.

11. Common Use Cases

- Machine translation
- Text summarization
- Speech-to-text
- Image captioning
- Code generation

- Question answering

12. Common Mistakes

- Not shifting decoder inputs and targets correctly
- Ignoring start/end tokens
- Evaluating only token accuracy for generation
- Greedy decoding when beam search is needed
- Training without masks for padding

13. Edge Cases / Limitations

The fixed context vector bottleneck hurts long inputs. Autoregressive decoding can accumulate errors. Beam search can produce generic outputs.

14. Variations

Variation	What changes	When to use	Importance
RNN seq2seq	RNN encoder and decoder	Small sequence tasks	Medium
Attention seq2seq	Decoder attends encoder states	Translation/summarization	High
Transformer encoder-decoder	Self-attention and cross-attention	Modern NLP	High
CNN encoder-decoder	Conv encoder/decoder	Segmentation, image translation	High

15. Related Topics

Attention improves encoder-decoder by removing fixed-vector bottlenecks. Transformers are modern encoder-decoder models. Autoencoders are encoder-decoder models trained to reconstruct input.

16. Interview Questions

1. What is encoder-decoder architecture?
A model that encodes input and decodes output.
2. Why is it used for translation?
It maps one sequence to another with different length.
3. What is teacher forcing?
Feeding true previous target tokens during training.
4. What is exposure bias?
Training uses true previous tokens, inference uses generated ones.
5. Why add attention?
To access all encoder states instead of one bottleneck vector.
6. What is beam search?
Decoding that keeps top candidate sequences.

7. What is autoregressive decoding?
Generating each token conditioned on previous generated tokens.
8. How are targets shifted?
Decoder input starts with BOS; labels end with EOS.
9. What metrics are used for summarization?
ROUGE, BLEU, BERTScore, human evaluation.
10. Encoder-decoder vs decoder-only?
Encoder-decoder explicitly conditions on source; decoder-only predicts next tokens in one stream.

17. Practice Tasks

- Build toy English-to-reversed-English seq2seq.
- Add attention to an RNN encoder-decoder.
- Implement greedy decoding.
- Compare teacher forcing ratios.
- Evaluate translation with BLEU.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Text Summarizer	Generates short summaries	PyTorch/HF	CNN-DailyMail	NLP generation
Image Captioner	Describes images	CNN + GRU	Flickr8k/MSCOCO	Multimodal project
Code Comment Generator	Generates comments from code	Transformers	CodeSearchNet	AI engineering relevance

19. Quick Revision

- Key idea: encode input, decode output.
- Main formula: $P(y|x) = \prod P(y_t | y_{<t}, x)$.
- When to use: sequence-to-sequence tasks.
- Important metrics: BLEU, ROUGE, exact match.
- Common traps: wrong shifting, no masks.
- Interview one-liner: encoder-decoder models learn conditional generation.

20. Final Cheat Sheet

Item	Answer
Definition	Input encoder plus output decoder
Input/output	Source sequence to target sequence
Main steps	Encode, decode autoregressively, train with CE
Key hyperparameters	Hidden size, layers, beam width, max length
Metrics	BLEU, ROUGE, EM

Item	Answer
Pros	Handles variable input/output lengths
Cons	Decoding is slow and errors accumulate
Best use cases	Translation, summarization, captioning

Attention

1. Overview

Attention is a mechanism that lets a model focus on relevant parts of the input when producing a representation or output. It is a core building block of Transformers and modern NLP, vision, speech, and multimodal AI.

2. Intuition

When answering "Who wrote Hamlet?", you focus on "Hamlet" and connect it to "Shakespeare." Attention lets each token look at other tokens and decide which ones matter.

3. Prerequisites

- Vectors and dot products
- Softmax
- Embeddings
- Sequence modeling
- Matrix multiplication
- Encoder-decoder basics

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Query	What a position is looking for	Drives matching	Word asking for context	Q/K/V roles
Key	What each position offers for matching	Compared with query	Token identity	Dot-product score
Value	Information retrieved	Weighted sum output	Token meaning vector	Why separate K and V
Softmax weights	Normalized importance	Makes weighted average	Attend 70% to one token	Interpretability limits
Self-attention	Tokens attend within same sequence	Contextual embeddings	BERT/GPT	Self vs cross attention

Subtopic	Meaning	Why it matters	Example	Interview angle
Cross-attention	Decoder attends encoder	Source conditioning	Translation	Encoder-decoder Transformer

5. Algorithm / Working Process

Create query, key, and value vectors from inputs. Compute similarity between queries and keys. Scale scores, apply mask if needed, softmax into attention weights, then compute weighted sum of values.

6. Mathematical Foundation

Scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

The division by $\sqrt{d_k}$ prevents large dot products from making softmax too sharp.

7. Practical Implementation

```
import torch
import torch.nn.functional as F

def scaled_dot_product_attention(q, k, v, mask=None):
    scores = q @ k.transpose(-2, -1) / (q.size(-1) ** 0.5)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float("-inf"))
    weights = F.softmax(scores, dim=-1)
    return weights @ v, weights

q = torch.randn(2, 5, 64)
k = torch.randn(2, 5, 64)
v = torch.randn(2, 5, 64)
context, weights = scaled_dot_product_attention(q, k, v)
print(context.shape, weights.shape)
```

8. Code Explanation

$q @ k.transpose(-2, -1)$ computes all pairwise token similarities. Softmax converts scores into weights. Multiplying weights by v aggregates information from relevant positions.

9. Training / Evaluation

Attention is trained end-to-end inside a larger model. Evaluate the final task: accuracy, F1, BLEU, ROUGE, perplexity, mAP, or IoU. Use masks for padding and causal generation.

10. Complexity and Cost

Self-attention over sequence length n costs $O(n^2 * d)$ time and $O(n^2)$ memory for attention weights. This is the main bottleneck for long-context models.

11. Common Use Cases

- Machine translation
- LLMs
- Document understanding
- Vision Transformers
- Speech recognition
- Multimodal retrieval

12. Common Mistakes

- Forgetting the causal mask in language models
- Not masking padding tokens
- Misunderstanding attention weights as full explanations
- Shape errors in multi-head attention
- Missing scale factor $\sqrt{d_k}$

13. Edge Cases / Limitations

Quadratic cost makes long sequences expensive. Attention can focus on spurious correlations. It does not automatically imply causal explanation.

14. Variations

Variation	What changes	When to use	Importance
Additive attention	MLP scoring	Older seq2seq	Medium
Dot-product attention	Dot similarity	Efficient attention	High
Multi-head attention	Multiple attention subspaces	Transformers	High
Sparse attention	Attend to selected positions	Long sequences	Medium
FlashAttention	Memory-efficient exact attention	Fast training/inference	High

15. Related Topics

Attention is the central operation in Transformers. Cross-attention connects encoder-decoder models. Self-attention competes with recurrence in RNN/LSTM/GRU.

16. Interview Questions

1. What is attention?
A weighted information retrieval mechanism over tokens/features.
2. What are Q, K, and V?
Query asks, key matches, value provides content.

3. Why scale by $\sqrt{d_k}$?
To keep softmax gradients stable.
4. What is self-attention?
Attention where Q, K, and V come from the same sequence.
5. What is cross-attention?
Attention where queries attend to another sequence's keys/values.
6. Why use masks?
To ignore padding or prevent looking at future tokens.
7. What is multi-head attention?
Several attention operations run in parallel.
8. Why is attention expensive for long sequences?
It computes all pairwise token interactions.
9. Are attention weights explanations?
Not always; they are useful signals but not guaranteed causal explanations.
10. Attention vs RNN?
Attention directly connects positions; RNN passes information through hidden states.

17. Practice Tasks

- Implement scaled dot-product attention from scratch.
- Add causal masking.
- Visualize attention weights for a sentence.
- Compare attention and mean pooling for classification.
- Profile attention memory as sequence length grows.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Attention Visualizer	Shows token attention maps	PyTorch, Streamlit	Any text	Great interview demo
Mini Translator	Seq2seq with attention	PyTorch	Multi30k	NLP fundamentals
Document Classifier	Uses attention pooling	PyTorch	AG News	Practical NLP

19. Quick Revision

- Key idea: weighted retrieval from relevant positions.
- Main formula: $\text{softmax}(QK^T/\sqrt{d_k})V$.
- When to use: contextual sequence/image modeling.
- Important metrics: task-specific.
- Common traps: missing masks, shape errors.
- Interview one-liner: attention lets each token dynamically choose useful context.

20. Final Cheat Sheet

Item	Answer
Definition	Weighted context aggregation
Input/output	Q/K/V tensors to context vectors
Main steps	Score, scale, mask, softmax, weighted sum
Key hyperparameters	Heads, head dim, dropout, context length
Metrics	Depends on downstream task
Pros	Captures long-range dependencies
Cons	Quadratic cost
Best use cases	Transformers, seq2seq, multimodal models

Transformer

1. Overview

The Transformer is a neural architecture based on self-attention, feedforward layers, residual connections, normalization, and positional information. It is the foundation of modern LLMs, BERT-style encoders, translation systems, Vision Transformers, speech models, and multimodal AI.

2. Intuition

A Transformer lets every token talk to every other token in parallel. Instead of reading a sentence word by word like an RNN, it builds contextual meaning by comparing all tokens at once.

3. Prerequisites

- Attention and Q/K/V
- Embeddings
- Positional encoding
- LayerNorm
- Residual connections
- Cross-entropy and language modeling

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Self-attention	Token-token interaction	Captures context	"bank" depends on nearby words	Formula
Multi-head attention	Multiple attention views	Learns different relations	syntax + coreference	Why heads

Subtopic	Meaning	Why it matters	Example	Interview angle
Positional encoding	Adds order info	Attention alone is permutation-invariant	token index	Sinusoidal vs learned
Feedforward block	Per-token MLP	Adds non-linear transformation	expand 768 to 3072	Why same for each token
Residual connection	Adds input to output	Helps gradients	$x + \text{block}(x)$	Deep training
LayerNorm	Normalizes features	Stabilizes training	pre-norm Transformer	BatchNorm vs LayerNorm

5. Algorithm / Working Process

Input tokens become embeddings. Positional information is added. Each Transformer block applies self-attention, residual connection, normalization, feedforward network, another residual connection, and normalization. Encoder-only models output contextual embeddings. Decoder-only models predict next tokens using causal masks. Encoder-decoder models use cross-attention for conditional generation.

6. Mathematical Foundation

Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

Transformer block, simplified:

```
x = x + MHA(LayerNorm(x))
x = x + FFN(LayerNorm(x))
```

Language modeling loss:

$$L = -\sum_t \log P(x_t \mid x_{<t})$$

7. Practical Implementation

```
import torch
from torch import nn

class TinyTransformerClassifier(nn.Module):
    def __init__(self, vocab_size, classes=2, d_model=128, heads=4, layers=2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model)
```

```

self.pos = nn.Parameter(torch.randn(1, 128, d_model))
block = nn.TransformerEncoderLayer(
    d_model=d_model,
    nhead=heads,
    dim_feedforward=4 * d_model,
    batch_first=True,
)
self.encoder = nn.TransformerEncoder(block, num_layers=layers)
self.head = nn.Linear(d_model, classes)

def forward(self, tokens):
    x = self.embedding(tokens) + self.pos[:, :tokens.size(1)]
    x = self.encoder(x)
    return self.head(x[:, 0])

model = TinyTransformerClassifier(vocab_size=10000)
tokens = torch.randint(0, 10000, (8, 64))
print(model(tokens).shape)

```

8. Code Explanation

The embedding layer maps token IDs to vectors. The learned positional parameter gives the model order information. `TransformerEncoderLayer` contains multi-head attention and feedforward layers. The first token representation is used as a classification summary.

9. Training / Evaluation

For classification, use labeled text and cross-entropy. For language modeling, use next-token prediction and perplexity. Use attention masks for padding and causal masks for generation. Important hyperparameters are model dimension, heads, layers, context length, learning rate schedule, warmup, and weight decay.

10. Complexity and Cost

Self-attention cost is $O(n^2 d)$ for sequence length n and dimension d . Feedforward cost is often large too: $O(n d d_{ff})$. Transformers usually require GPUs/TPUs for large-scale training.

11. Common Use Cases

- LLMs and chatbots
- Translation and summarization
- Semantic search embeddings
- Code generation
- Vision Transformers
- Speech and multimodal models

12. Common Mistakes

- Forgetting positional encoding
- Missing causal mask in decoder
- Confusing encoder-only, decoder-only, and encoder-decoder

- Using too long context without memory planning
- Fine-tuning large models with too high learning rate

13. Edge Cases / Limitations

Transformers are data-hungry and expensive. Standard attention scales poorly with long sequences. They can hallucinate and may learn shortcuts from biased data.

14. Variations

Variation	What changes	When to use	Importance
Encoder-only	Bidirectional attention	Classification, embeddings	High
Decoder-only	Causal attention	LLM generation	High
Encoder-decoder	Cross-attention	Translation/summarization	High
Sparse Transformer	Reduced attention pattern	Long context	Medium
MoE Transformer	Sparse expert FFNs	Huge models efficiently	High

15. Related Topics

Transformer vs RNN: Transformers parallelize and model long context better. Transformer vs CNN: less local bias, more global interaction. Transformer vs Mamba: Transformers use attention; Mamba uses state space sequence modeling.

16. Interview Questions

1. What is a Transformer?
A neural architecture built mainly on self-attention and feedforward blocks.
2. Why are positional encodings needed?
Self-attention alone has no order awareness.
3. What is multi-head attention?
Attention run in several subspaces.
4. Encoder-only vs decoder-only?
Encoder-only reads bidirectionally; decoder-only predicts next tokens causally.
5. What is causal masking?
Preventing a token from attending to future tokens.
6. Why use residual connections?
They improve gradient flow and deep training.
7. Why LayerNorm instead of BatchNorm?
It works better for variable-length sequence features and small batches.
8. What is the main cost bottleneck?
Quadratic attention memory/time.
9. What is perplexity?
Exponentiated average negative log-likelihood for language models.
10. Why are Transformers powerful?
They combine parallel training with direct token-token interactions.

17. Practice Tasks

- Implement a tiny Transformer classifier.
- Add causal masking and train a character LM.
- Compare RNN and Transformer on text classification.
- Visualize attention heads.
- Fine-tune a Hugging Face model on a small dataset.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Resume Classifier	Categorizes resumes	Hugging Face, PyTorch	Resume dataset	Placement-relevant NLP
Mini GPT	Character-level generation	PyTorch	Shakespeare/text8	Shows fundamentals
FAQ Semantic Search	Embeds and retrieves FAQs	SentenceTransformers, FAISS	Company FAQs	AI engineering project

19. Quick Revision

- Key idea: self-attention plus feedforward blocks.
- Main formula: $\text{softmax}(QK^T/\sqrt{d_k})V$.
- When to use: language, vision, multimodal tasks.
- Important metrics: accuracy, F1, perplexity, BLEU/ROUGE.
- Common traps: masks and positional encoding.
- Interview one-liner: Transformers model contextual relationships through parallel self-attention.

20. Final Cheat Sheet

Item	Answer
Definition	Attention-based deep architecture
Input/output	Tokens/patches to contextual vectors/logits
Main steps	Embed, position, attention, FFN, output
Key hyperparameters	Layers, heads, d_{model} , context length
Metrics	Perplexity, F1, BLEU, ROUGE
Pros	Parallel, powerful, scalable
Cons	Expensive for long sequences
Best use cases	LLMs, NLP, ViT, multimodal AI

Autoencoder

1. Overview

An Autoencoder is an unsupervised neural network that learns to reconstruct its input through a compressed latent representation. It has an encoder, bottleneck, and decoder.

Autoencoders are used for dimensionality reduction, anomaly detection, denoising, representation learning, compression, and pretraining.

2. Intuition

It is like asking a student to summarize a long paragraph into a few keywords and then reconstruct the paragraph. Good reconstruction means the summary captured important information.

3. Prerequisites

- MLP/CNN basics
- Reconstruction loss
- Latent space
- Optimization
- Normalization

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Encoder	Compresses input	Learns representation	Image to vector	Feature learning
Bottleneck	Low-dimensional latent	Forces compression	784 to 32 dims	Undercomplete AE
Decoder	Reconstructs input	Tests latent usefulness	Vector to image	Reconstruction objective
Reconstruction loss	Difference between input/output	Training signal	MSE/BCE	Which loss to use
Denoising	Reconstruct clean from noisy	Robust features	Noisy image restoration	Denoising AE

5. Algorithm / Working Process

Input is passed through encoder to latent vector z . Decoder maps z back to reconstructed input $\hat{x}$. Training minimizes reconstruction error between x and $\hat{x}$. In anomaly detection, high reconstruction error suggests unusual input.

6. Mathematical Foundation

```
z = encoder(x)
x_hat = decoder(z)
```

$$L = ||x - x_{\text{hat}}||^2$$

For binary-like pixels:

$$L = -\sum_i [x_i \log(x_{\text{hat}_i}) + (1-x_i)\log(1-x_{\text{hat}_i})]$$

7. Practical Implementation

```
import torch
from torch import nn

class Autoencoder(nn.Module):
    def __init__(self, input_dim=784, latent_dim=32):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.ReLU(),
            nn.Linear(128, latent_dim),
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.ReLU(),
            nn.Linear(128, input_dim),
            nn.Sigmoid(),
        )

    def forward(self, x):
        z = self.encoder(x)
        return self.decoder(z)

model = Autoencoder()
x = torch.rand(16, 784)
x_hat = model(x)
loss = nn.MSELoss()(x_hat, x)
print(loss.item())
```

8. Code Explanation

The encoder reduces 784 input dimensions to 32 latent dimensions. The decoder reconstructs the original vector. `Sigmoid` keeps output in `[0,1]`, suitable for normalized image pixels.

9. Training / Evaluation

Train without labels using reconstruction loss. For anomaly detection, train mostly on normal data and choose a threshold from validation reconstruction errors. Evaluate with reconstruction MSE, anomaly ROC-AUC, precision-recall, or downstream task performance.

10. Complexity and Cost

Cost depends on encoder and decoder size. Dense autoencoders are cheap. Convolutional autoencoders for images cost more but preserve spatial structure better.

11. Common Use Cases

- Image denoising
- Anomaly detection
- Dimensionality reduction
- Feature learning
- Data compression

12. Common Mistakes

- Bottleneck too large, causing identity mapping
- Evaluating only reconstruction visually
- Training anomaly detector with many anomalies
- Wrong output activation for data scale
- Comparing MSE across differently normalized datasets

13. Edge Cases / Limitations

Autoencoders may reconstruct anomalies too well if overpowered. Latent space may not be smooth or generative unless regularized, unlike VAEs.

14. Variations

Variation	What changes	When to use	Importance
Denoising AE	Noisy input, clean target	Robust reconstruction	High
Sparse AE	Sparsity penalty	Interpretable features	Medium
Convolutional AE	CNN encoder/decoder	Images	High
Contractive AE	Penalizes sensitivity	Stable latent space	Low

15. Related Topics

Autoencoder vs PCA: both reduce dimensions; autoencoders learn non-linear mappings. Autoencoder vs VAE: VAE learns probabilistic latent distributions. U-Net is an encoder-decoder with skip connections.

16. Interview Questions

1. What is an autoencoder?
A model trained to reconstruct its input through a latent bottleneck.
2. Why use a bottleneck?
To force useful compressed representation learning.
3. Is it supervised?
Usually unsupervised or self-supervised.

4. What loss is common?
MSE for continuous data, BCE for normalized binary-like data.
5. How use AE for anomaly detection?
High reconstruction error indicates anomaly.
6. AE vs PCA?
AE can learn non-linear compression.
7. What happens if latent dimension is too large?
The model may learn identity mapping.
8. What is denoising AE?
It reconstructs clean input from corrupted input.
9. Is vanilla AE generative?
Not reliably; latent space is not explicitly regularized.
10. Why use convolutional AE for images?
It preserves spatial structure.

17. Practice Tasks

- Build MNIST autoencoder.
- Plot latent vectors with t-SNE.
- Train denoising autoencoder.
- Detect anomalies using reconstruction threshold.
- Compare PCA and autoencoder compression.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Industrial Anomaly AE	Flags defective images	PyTorch	MVTec AD	Strong CV anomaly project
Image Denoiser	Removes noise from images	PyTorch, OpenCV	MNIST/CIFAR	Clear visual demo
Tabular Compression	Learns compact features	PyTorch, sklearn	UCI datasets	Representation learning

19. Quick Revision

- Key idea: reconstruct input through bottleneck.
- Main formula: $L = ||x - \hat{x}||^2$.
- When to use: compression, denoising, anomaly detection.
- Important metrics: reconstruction loss, ROC-AUC.
- Common traps: oversized bottleneck.
- Interview one-liner: an autoencoder learns compressed representations by reconstructing its input.

20. Final Cheat Sheet

Item	Answer
Definition	Encoder-bottleneck-decoder reconstruction model
Input/output	Input to reconstructed input
Main steps	Encode, compress, decode, compare
Key hyperparameters	Latent dim, depth, loss, LR
Metrics	MSE, BCE, anomaly AUC
Pros	Unsupervised representation learning
Cons	Can learn identity mapping
Best use cases	Denoising, compression, anomaly detection

Variational Autoencoder

1. Overview

A Variational Autoencoder is a probabilistic generative autoencoder. Instead of encoding an input to a fixed latent vector, it encodes to a distribution, samples from that distribution, and decodes the sample.

VAEs are used for generative modeling, latent space interpolation, anomaly detection, representation learning, and controllable generation research.

2. Intuition

Vanilla autoencoders learn isolated points in latent space. VAEs shape latent space into a smooth cloud, so sampling nearby points produces meaningful outputs.

3. Prerequisites

- Autoencoders
- Gaussian distributions
- KL divergence
- Reparameterization trick
- Reconstruction loss
- Basic probability

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Latent distribution	Encoder outputs mean and variance	Enables sampling	<code>mu, logvar</code>	AE vs VAE

Subtopic	Meaning	Why it matters	Example	Interview angle
Reparameterization	Sampling in differentiable form	Allows backprop	$z = \mu + \sigma * \epsilon$	Why needed
KL divergence	Regularizes latent distribution	Keeps latent near normal	Match $N(0, I)$	ELBO
Reconstruction term	Keeps outputs accurate	Prevents useless latent	MSE/BCE	Balance with KL
Generative sampling	Decode random latent	Creates new data	Sample digit	Why VAE is generative

5. Algorithm / Working Process

Encoder maps input to μ and $\log\sigma$. Sample ϵ from standard normal. Compute $z = \mu + \sigma * \epsilon$. Decoder reconstructs input from z . Training minimizes reconstruction loss plus KL divergence.

6. Mathematical Foundation

Evidence lower bound:

$$\text{ELBO} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \text{KL}(q(z|x) || p(z))$$

Loss minimized:

```
L = reconstruction_loss + KL(q(z|x) || N(0,I))
KL = -0.5 * sum(1 + log(sigma^2) - mu^2 - sigma^2)
z = mu + sigma * epsilon, epsilon ~ N(0,I)
```

7. Practical Implementation

```
import torch
from torch import nn
import torch.nn.functional as F

class VAE(nn.Module):
    def __init__(self, input_dim=784, latent_dim=20):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, 400)
        self.mu = nn.Linear(400, latent_dim)
        self.logvar = nn.Linear(400, latent_dim)
        self.fc2 = nn.Linear(latent_dim, 400)
        self.fc3 = nn.Linear(400, input_dim)
```

```
def encode(self, x):
    h = F.relu(self.fc1(x))
    return self.mu(h), self.logvar(h)

def reparameterize(self, mu, logvar):
    std = torch.exp(0.5 * logvar)
    eps = torch.randn_like(std)
    return mu + std * eps

def decode(self, z):
    return torch.sigmoid(self.fc3(F.relu(self.fc2(z))))

def forward(self, x):
    mu, logvar = self.encode(x)
    z = self.reparameterize(mu, logvar)
    return self.decode(z), mu, logvar

def vae_loss(x_hat, x, mu, logvar):
    recon = F.binary_cross_entropy(x_hat, x, reduction="sum")
    kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return recon + kl
```

8. Code Explanation

The encoder outputs `mu` and `logvar`. The reparameterization trick separates randomness from trainable parameters. The loss combines reconstruction quality and latent regularization.

9. Training / Evaluation

Use normalized inputs. Track reconstruction loss and KL term separately. Evaluate sample quality visually, reconstruction error, latent interpolation, downstream classification, or anomaly detection AUC. Tune latent dimension and KL weight.

10. Complexity and Cost

Similar to autoencoders plus extra mean/logvar layers and sampling. VAEs are cheaper than GANs/diffusion models but often produce blurrier images.

11. Common Use Cases

- Generating simple images
- Latent interpolation
- Anomaly detection
- Semi-supervised learning
- Representation learning

12. Common Mistakes

- Forgetting KL term
- Sampling directly without reparameterization

- Posterior collapse
- Wrong reconstruction loss for data type
- KL term overpowering reconstruction

13. Edge Cases / Limitations

VAEs often produce blurry samples with pixel-wise losses. Posterior collapse can occur, especially with powerful decoders.

14. Variations

Variation	What changes	When to use	Importance
Beta-VAE	Weighted KL term	Disentanglement	Medium
Conditional VAE	Conditions on labels/text	Controlled generation	High
VQ-VAE	Discrete latent codes	Image/audio tokenization	High
Hierarchical VAE	Multiple latent levels	Rich generation	Research

15. Related Topics

VAE vs AE: VAE is probabilistic and generative. VAE vs GAN: VAE optimizes likelihood lower bound; GAN uses adversarial training. VQ-VAE connects to tokenizers for generative models.

16. Interview Questions

1. What is a VAE?
A probabilistic autoencoder trained with reconstruction plus KL loss.
2. Why output mean and variance?
To define a latent distribution.
3. What is the reparameterization trick?
 $z = \mu + \sigma * \epsilon$ to allow backprop through sampling.
4. What is KL divergence doing?
Regularizing latent distribution toward prior.
5. What is ELBO?
A lower bound on log-likelihood optimized by VAE.
6. Why are VAEs generative?
You can sample latent z from prior and decode it.
7. What is posterior collapse?
Decoder ignores latent variable.
8. VAE vs GAN?
VAE has explicit probabilistic objective; GAN learns through discriminator.
9. Why might VAE images be blurry?
Pixel reconstruction losses average possible outputs.
10. What is beta-VAE?
VAE with weighted KL term for disentanglement.

17. Practice Tasks

- Train VAE on MNIST.
- Plot 2D latent space.
- Generate samples from random z .
- Experiment with beta values.
- Detect anomalies with reconstruction plus KL score.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Digit Generator	Samples handwritten digits	PyTorch	MNIST	Generative basics
Fashion Latent Explorer	Interpolates clothing images	PyTorch, Streamlit	Fashion-MNIST	Visual portfolio
VAE Anomaly Detector	Finds abnormal records	PyTorch	Credit fraud/MVTec	Practical anomaly detection

19. Quick Revision

- Key idea: learn smooth probabilistic latent space.
- Main formula: $L = \text{recon} + \text{KL}(q(z|x) || N(\theta, I))$.
- When to use: generative representation learning.
- Important metrics: recon loss, KL, sample quality.
- Common traps: no reparameterization, posterior collapse.
- Interview one-liner: VAE makes autoencoders generative by encoding distributions, not points.

20. Final Cheat Sheet

Item	Answer
Definition	Probabilistic generative autoencoder
Input/output	Input to reconstruction plus latent distribution
Main steps	Encode mu/logvar, sample, decode, KL+recon loss
Key hyperparameters	Latent dim, beta/KL weight, LR
Metrics	ELBO, recon loss, anomaly AUC
Pros	Smooth latent sampling
Cons	Blurry samples, posterior collapse
Best use cases	Latent generation and anomaly detection

ResNet

1. Overview

ResNet, or Residual Network, is a deep CNN architecture that uses skip connections to train very deep networks effectively. It was a major breakthrough in computer vision.

ResNets are used as backbones for classification, detection, segmentation, medical imaging, satellite vision, and feature extraction.

2. Intuition

Instead of forcing each block to learn a complete transformation, ResNet lets it learn a correction to the input. If a layer is not needed, it can learn near-zero residual and pass information through.

3. Prerequisites

- CNNs
- Backpropagation
- Vanishing gradients
- Batch normalization
- Residual connections

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Skip connection	Adds input to block output	Improves gradient flow	$y = F(x) + x$	Why ResNet trains deep
Residual function	Learns correction	Easier optimization	edge refinement	Residual learning
Basic block	Two 3x3 convs	ResNet-18/34	simple residual block	Architecture difference
Bottleneck block	1x1, 3x3, 1x1 convs	Efficient deep ResNets	ResNet-50	Why bottleneck
Projection shortcut	Matches dimensions	Needed when channels/size change	1x1 conv skip	Shape compatibility

5. Algorithm / Working Process

Input image passes through initial conv and pooling. Residual stages apply blocks. Each block computes $F(x)$ and adds shortcut x . Spatial size usually decreases stage by stage while channels increase. Global average pooling and linear layer produce logits.

6. Mathematical Foundation

Residual block:

$$y = F(x, W) + x$$

If dimensions differ:

$$y = F(x, W) + W_s \cdot x$$

Gradient benefit:

$$dL/dx = dL/dy * (dF/dx + I)$$

The identity term helps gradients flow backward.

7. Practical Implementation

```
import torch
from torch import nn

class BasicBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(channels, channels, 3, padding=1, bias=False),
            nn.BatchNorm2d(channels),
            nn.ReLU(),
            nn.Conv2d(channels, channels, 3, padding=1, bias=False),
            nn.BatchNorm2d(channels),
        )
        self.relu = nn.ReLU()

    def forward(self, x):
        return self.relu(self.block(x) + x)

x = torch.randn(4, 64, 32, 32)
block = BasicBlock(64)
print(block(x).shape)
```

8. Code Explanation

The block applies two convolution layers and adds the original input before final ReLU. Because channels and spatial size stay the same, no projection shortcut is needed.

9. Training / Evaluation

Use standard image augmentation, normalization, SGD/AdamW, weight decay, and learning rate schedules. Metrics include top-1/top-5 accuracy, F1, mAP for detection, and IoU for segmentation when ResNet is used as backbone.

10. Complexity and Cost

Cost depends on depth: ResNet-18 is lightweight, ResNet-50 is common, ResNet-101/152 are heavier. Residual connections add little compute but greatly improve optimization.

11. Common Use Cases

- Image classification
- Object detection backbone
- Segmentation encoder
- Medical image feature extractor
- Transfer learning

12. Common Mistakes

- Adding tensors with mismatched shapes
- Forgetting projection shortcut when channels change
- Training very deep CNN without normalization
- Using ImageNet normalization incorrectly
- Fine-tuning all layers on tiny data without regularization

13. Edge Cases / Limitations

ResNets still have limited global context compared with attention models. Large ResNets can be costly for mobile deployment.

14. Variations

Variation	What changes	When to use	Importance
ResNet-18/34	Basic blocks	Faster tasks	High
ResNet-50/101	Bottleneck blocks	Stronger accuracy	High
ResNeXt	Grouped convolutions	Better accuracy/efficiency	Medium
Wide ResNet	Wider channels	CIFAR-style tasks	Medium

15. Related Topics

ResNet is a CNN architecture. U-Net often uses residual encoders. Transformers also use residual connections. DenseNet extends feature reuse by concatenation instead of addition.

16. Interview Questions

1. What problem did ResNet solve?
Training degradation in very deep networks.
2. What is a residual connection?
Adding block input to block output.
3. Why does $F(x)+x$ help?
It improves gradient flow and lets blocks learn corrections.

4. When need projection shortcut?
When channel count or spatial size changes.
5. Basic block vs bottleneck?
Basic uses 3x3 convs; bottleneck uses 1x1-3x3-1x1.
6. Why use global average pooling?
It reduces parameters compared with flattening.
7. ResNet vs plain CNN?
ResNet trains much deeper models.
8. What is degradation problem?
Deeper plain networks perform worse despite more capacity.
9. Is skip connection only for CNNs?
No, Transformers also use residual connections.
10. Why is ResNet popular for transfer learning?
Strong pretrained visual features.

17. Practice Tasks

- Implement a residual block.
- Train ResNet-18 with transfer learning.
- Compare plain CNN vs ResNet on CIFAR-10.
- Add projection shortcut for downsampling.
- Visualize feature maps.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Medical X-ray Classifier	Detects disease	PyTorch, torchvision	ChestX-ray14	Healthcare CV
Product Image Classifier	Categorizes products	PyTorch	E-commerce images	Industry CV
Transfer Learning Benchmark	Compares ResNet variants	PyTorch	CIFAR/Food-101	Strong experimentation

19. Quick Revision

- Key idea: learn residual corrections with skip connections.
- Main formula: $y = F(x) + x$.
- When to use: deep CNN backbones.
- Important metrics: accuracy, mAP, IoU.
- Common traps: shortcut shape mismatch.
- Interview one-liner: ResNet made very deep CNNs trainable using identity shortcuts.

20. Final Cheat Sheet

Item	Answer
Definition	CNN with residual skip connections
Input/output	Image to features/logits
Main steps	Conv block plus shortcut addition
Key hyperparameters	Depth, block type, channels
Metrics	Accuracy, mAP, IoU
Pros	Trains deep models well
Cons	Less global context than attention
Best use cases	Vision backbones and transfer learning

U-Net

1. Overview

U-Net is an encoder-decoder CNN architecture for dense prediction, especially image segmentation. It has a contracting path for context and an expanding path for precise localization, connected by skip connections.

It is widely used in biomedical segmentation, satellite imagery, road extraction, industrial inspection, and image-to-image tasks.

2. Intuition

The encoder asks "what is in the image?", while the decoder asks "where exactly is it?" Skip connections pass high-resolution details from encoder to decoder.

3. Prerequisites

- CNNs
- Encoder-decoder architecture
- Upsampling/transposed convolution
- Segmentation masks
- Dice/IoU metrics

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Contracting path	Downsampling encoder	Captures context	Organ shape	Why downsample
Expanding path	Upsampling decoder	Restores resolution	Pixel mask	Transposed conv
Skip connections	Copy encoder features	Preserves boundaries	Edge details	U-Net vs autoencoder

Subtopic	Meaning	Why it matters	Example	Interview angle
Segmentation mask	Per-pixel label	Dense prediction output	tumor mask	Loss and metrics
Dice loss	Overlap-based loss	Handles imbalance	small lesion	Dice vs IoU

5. Algorithm / Working Process

Input image goes through encoder blocks with pooling. Decoder upsamples features step by step. At each decoder stage, it concatenates corresponding encoder features. Final 1x1 convolution predicts class logits for every pixel.

6. Mathematical Foundation

Binary Dice coefficient:

$$\text{Dice} = \frac{2|P \text{ intersect } Y|}{(|P| + |Y|)}$$

$$\text{DiceLoss} = 1 - \text{Dice}$$

Pixel-wise cross-entropy:

$$L = -\sum_{\text{pixels}} \sum_{\text{classes}} y_c \log(p_c)$$

7. Practical Implementation

```
import torch
from torch import nn

def conv_block(cin, cout):
    return nn.Sequential(
        nn.Conv2d(cin, cout, 3, padding=1),
        nn.ReLU(),
        nn.Conv2d(cout, cout, 3, padding=1),
        nn.ReLU(),
    )

class MiniUNet(nn.Module):
    def __init__(self, classes=1):
        super().__init__()
        self.enc1 = conv_block(3, 32)
        self.pool = nn.MaxPool2d(2)
        self.enc2 = conv_block(32, 64)
        self.up = nn.ConvTranspose2d(64, 32, 2, stride=2)
        self.dec1 = conv_block(64, 32)
        self.out = nn.Conv2d(32, classes, 1)
```

```
def forward(self, x):
    e1 = self.enc1(x)
    e2 = self.enc2(self.pool(e1))
    d1 = self.up(e2)
    d1 = torch.cat([d1, e1], dim=1)
    return self.out(self.dec1(d1))

model = MiniUNet()
print(model(torch.randn(2, 3, 128, 128)).shape)
```

8. Code Explanation

`enc1` keeps high-resolution features. `pool` downsamples. `ConvTranspose2d` upsamples. `torch.cat` joins decoder and encoder features. The final `1x1` convolution predicts per-pixel logits.

9. Training / Evaluation

Prepare image-mask pairs with identical geometric augmentations. Use BCE/Dice for binary segmentation or cross-entropy/Dice for multi-class. Metrics include IoU, Dice, pixel accuracy, precision, and recall. Use patch training for large images.

10. Complexity and Cost

Memory is high because skip features must be stored. Training segmentation models on large images often needs GPUs and careful batch sizing.

11. Common Use Cases

- Medical image segmentation
- Satellite land-cover mapping
- Road/lane segmentation
- Cell/nuclei segmentation
- Defect localization

12. Common Mistakes

- Applying different augmentations to image and mask
- Using bilinear interpolation for class masks
- Shape mismatch during concatenation
- Ignoring class imbalance
- Evaluating only pixel accuracy

13. Edge Cases / Limitations

Small objects and fuzzy boundaries are difficult. U-Net may struggle with global context unless enhanced with attention or pretrained encoders.

14. Variations

Variation	What changes	When to use	Importance
ResUNet	Residual blocks	Deeper segmentation	High
Attention U-Net	Attention gates	Focus on target regions	Medium
U-Net++	Nested skip paths	Better fine detail	Medium
3D U-Net	Volumetric convs	CT/MRI volumes	High

15. Related Topics

U-Net vs autoencoder: U-Net keeps skip connections for spatial detail. U-Net vs FCN: U-Net has symmetric encoder-decoder with strong skips. ResNet can be used as U-Net encoder.

16. Interview Questions

1. What is U-Net used for?
Image segmentation and dense prediction.
2. Why skip connections?
They recover fine spatial details.
3. What is Dice coefficient?
Overlap metric between prediction and ground truth.
4. Why not use pixel accuracy alone?
It can be misleading with class imbalance.
5. What does 1x1 conv do at output?
Maps features to class logits per pixel.
6. What is transposed convolution?
Learnable upsampling.
7. Why is U-Net popular in medical imaging?
It works well with limited data and precise masks.
8. How handle multi-class segmentation?
Output C channels and use cross-entropy/Dice.
9. What causes mask augmentation bugs?
Different transforms or wrong interpolation.
10. U-Net vs ResNet?
U-Net is segmentation encoder-decoder; ResNet is usually classification backbone.

17. Practice Tasks

- Segment cells from microscopy images.
- Implement Dice loss.
- Debug mask interpolation artifacts.
- Train U-Net with a ResNet encoder.
- Compare BCE and Dice loss.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Tumor Segmentation	Segments tumors	PyTorch	BraTS	Healthcare AI
Road Extraction	Finds roads in satellite images	PyTorch, OpenCV	DeepGlobe	Geospatial CV
Defect Localizer	Produces defect masks	PyTorch	MVTec AD	Industrial inspection

19. Quick Revision

- Key idea: encoder-decoder segmentation with skip connections.
- Main formula: $\text{Dice} = 2 \cdot \text{intersection} / (\text{pred} + \text{target})$.
- When to use: pixel-level prediction.
- Important metrics: Dice, IoU.
- Common traps: mask augmentation and imbalance.
- Interview one-liner: U-Net combines context and localization for segmentation.

20. Final Cheat Sheet

Item	Answer
Definition	CNN encoder-decoder for segmentation
Input/output	Image to pixel mask
Main steps	Downsample, upsample, concatenate skips
Key hyperparameters	Depth, channels, loss, image size
Metrics	Dice, IoU, pixel F1
Pros	Excellent localization
Cons	High memory, limited global context
Best use cases	Medical and satellite segmentation

Vision Transformer

1. Overview

Vision Transformer applies the Transformer architecture to images by splitting images into patches and treating those patches like tokens. It uses self-attention to model relationships between image regions.

ViTs are used in image classification, detection, segmentation, multimodal models, medical imaging, and self-supervised visual pretraining.

2. Intuition

Instead of scanning filters like CNNs, ViT cuts an image into patches like puzzle pieces. Each patch attends to other patches to understand the full image.

3. Prerequisites

- Transformer architecture
- Image tensors and patches
- Positional embeddings
- Classification tokens
- Transfer learning

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Patch embedding	Converts patches to tokens	Bridges image to Transformer	16x16 patch vector	Patch size tradeoff
CLS token	Learnable summary token	Used for classification	first token output	BERT similarity
Positional embedding	Patch position info	Preserves spatial order	row/column location	Why needed
Self-attention	Patch interactions	Captures global context	object parts relate	ViT vs CNN
Pretraining	Large data training	ViT needs data	ImageNet-21k	Data hunger

5. Algorithm / Working Process

Image is split into fixed-size patches. Each patch is flattened and linearly projected to an embedding. Add positional embeddings and optional CLS token. Transformer encoder processes patch tokens. CLS output or pooled patch output goes to classifier.

6. Mathematical Foundation

Number of patches:

$$N = (H/P) * (W/P)$$

Patch projection:

$$z_i = x_{\text{patch}_i} E + \text{pos}_i$$

Attention cost:

$O(N^2 d)$

Smaller patch size increases N and attention cost.

7. Practical Implementation

```
import torch
from torch import nn

class TinyViT(nn.Module):
    def __init__(self, image_size=32, patch=4, classes=10, d=128):
        super().__init__()
        patches = (image_size // patch) ** 2
        self.patch = nn.Conv2d(3, d, kernel_size=patch, stride=patch)
        self.cls = nn.Parameter(torch.zeros(1, 1, d))
        self.pos = nn.Parameter(torch.randn(1, patches + 1, d))
        layer = nn.TransformerEncoderLayer(d, nhead=4, batch_first=True)
        self.encoder = nn.TransformerEncoder(layer, num_layers=2)
        self.head = nn.Linear(d, classes)

    def forward(self, x):
        x = self.patch(x).flatten(2).transpose(1, 2)
        cls = self.cls.expand(x.size(0), -1, -1)
        x = torch.cat([cls, x], dim=1) + self.pos
        return self.head(self.encoder(x)[: , 0])
```

8. Code Explanation

`Conv2d` with kernel and stride equal to patch size performs patch embedding efficiently. The CLS token is prepended. Positional embeddings are added before Transformer encoding.

9. Training / Evaluation

ViTs usually need large datasets or pretrained weights. Use strong augmentation, regularization, AdamW, warmup, and cosine decay. Evaluate with accuracy, top-k accuracy, F1, mAP, or IoU depending on downstream task.

10. Complexity and Cost

Attention cost grows quadratically with number of patches. A 224x224 image with 16x16 patches has 196 patches; smaller patches increase cost quickly.

11. Common Use Cases

- Image classification
- Visual representation learning
- Multimodal models like CLIP-style systems

- Detection/segmentation backbones
- Medical image analysis

12. Common Mistakes

- Training ViT from scratch on tiny data
- Forgetting positional embeddings
- Choosing patch size too small for compute budget
- Ignoring image normalization
- Comparing ViT and CNN without equivalent pretraining

13. Edge Cases / Limitations

ViT has weaker local inductive bias than CNN and can be data-hungry. High-resolution images are expensive due to patch attention.

14. Variations

Variation	What changes	When to use	Importance
DeiT	Data-efficient training tricks	Smaller datasets	High
Swin Transformer	Windowed hierarchical attention	Detection/segmentation	High
MAE	Masked autoencoding pretraining	Self-supervised vision	High
Hybrid ViT	CNN stem plus Transformer	Better local bias	Medium

15. Related Topics

ViT vs CNN: ViT uses global attention over patches; CNN uses local filters. ViT is a Transformer adapted to images. MAE is an autoencoder-like self-supervised ViT method.

16. Interview Questions

1. How does ViT process images?
It splits images into patches and treats patches as tokens.
2. Why positional embeddings?
Patch tokens need location information.
3. What is patch size tradeoff?
Smaller patches capture detail but increase attention cost.
4. What is CLS token?
A learnable summary token for classification.
5. ViT vs CNN?
ViT has global attention; CNN has local inductive bias.
6. Why does ViT need more data?
It has less built-in image bias.
7. What is Swin Transformer?
A hierarchical ViT using shifted window attention.

8. Can ViT do segmentation?
Yes, with decoder/segmentation heads.
9. What optimizer is common?
AdamW with learning rate schedule.
10. What is MAE?
Masked Autoencoder pretraining for ViTs.

17. Practice Tasks

- Implement patch embedding.
- Fine-tune pretrained ViT.
- Compare patch sizes.
- Visualize attention maps.
- Train CNN and ViT on CIFAR-10.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
ViT Classifier	Classifies images	PyTorch/HF	Food-101	Modern CV
Medical ViT	Detects disease	PyTorch	Chest X-ray	Research-oriented
CLIP-like Retrieval	Finds matching images/text	PyTorch	Flickr30k	Multimodal AI

19. Quick Revision

- Key idea: image patches as Transformer tokens.
- Main formula: $N = (H/P) * (W/P)$.
- When to use: large-scale vision/pretrained models.
- Important metrics: accuracy, mAP, IoU.
- Common traps: tiny data from scratch.
- Interview one-liner: ViT replaces convolutional locality with global patch self-attention.

20. Final Cheat Sheet

Item	Answer
Definition	Transformer for image patches
Input/output	Image to class/features
Main steps	Patchify, embed, add positions, Transformer
Key hyperparameters	Patch size, depth, heads, d_model
Metrics	Accuracy, top-k, mAP
Pros	Global context, scalable pretraining
Cons	Data/compute hungry
Best use cases	Pretrained modern vision systems

Graph Neural Network

1. Overview

A Graph Neural Network learns from graph-structured data made of nodes and edges. It updates node representations by aggregating information from neighbors.

GNNs are used in social networks, recommendation systems, fraud detection, molecules, knowledge graphs, traffic prediction, and program analysis.

2. Intuition

To understand a person in a social network, look not only at their profile but also their friends and friends' behavior. GNNs repeatedly pass messages across graph edges.

3. Prerequisites

- Graph theory: nodes, edges, adjacency matrix
- Neural networks
- Matrix multiplication
- Aggregation functions
- Semi-supervised learning

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Node features	Attributes of nodes	Initial information	user age, molecule atom type	Feature design
Edge index/adjacency	Graph connections	Defines message paths	friendship edge	Sparse representation
Message passing	Neighbor information exchange	Core GNN operation	aggregate friends	Oversmoothing
Aggregation	Sum/mean/max/attention	Handles variable neighbors	mean neighbor embedding	Permutation invariance
Readout	Graph-level pooling	Whole graph prediction	molecule toxicity	Node vs graph task

5. Algorithm / Working Process

Initialize node embeddings from features. For each layer, each node receives messages from neighbors, aggregates them, combines with its own state, and applies non-linearity. For node tasks, output per node. For

graph tasks, pool node embeddings and classify/regress.

6. Mathematical Foundation

GCN update:

$$H^{l+1} = \text{sigma}(D^{-1/2} \hat{A} D^{-1/2} H^l W^l)$$

where $\hat{A} = A + I$ includes self-loops and D is degree matrix.

Generic message passing:

```
m_v = aggregate({h_u : u in N(v)})
h_v' = update(h_v, m_v)
```

7. Practical Implementation

```
import torch
from torch import nn

class SimpleGCNLayer(nn.Module):
    def __init__(self, in_dim, out_dim):
        super().__init__()
        self.linear = nn.Linear(in_dim, out_dim)

    def forward(self, x, adj):
        adj = adj + torch.eye(adj.size(0), device=adj.device)
        deg = adj.sum(dim=1)
        norm = adj / deg.clamp(min=1).unsqueeze(1)
        return torch.relu(self.linear(norm @ x))

class GCN(nn.Module):
    def __init__(self, in_dim, hidden, classes):
        super().__init__()
        self.g1 = SimpleGCNLayer(in_dim, hidden)
        self.g2 = SimpleGCNLayer(hidden, classes)

    def forward(self, x, adj):
        return self.g2(self.g1(x, adj), adj)
```

8. Code Explanation

The adjacency matrix defines neighbors. Adding identity creates self-loops. Degree normalization averages neighbor features. The linear layer transforms aggregated features.

9. Training / Evaluation

For node classification, split nodes into train/validation/test while keeping graph structure. For graph classification, split graphs. Metrics include accuracy, F1, ROC-AUC, Hits@K, MRR, or RMSE. Avoid leakage through graph splits.

10. Complexity and Cost

Sparse message passing costs roughly $O(E * d)$ per layer, where E is edge count. Dense adjacency costs $O(N^2)$ and is impractical for large graphs.

11. Common Use Cases

- Molecular property prediction
- Fraud ring detection
- Social recommendation
- Knowledge graph reasoning
- Traffic forecasting

12. Common Mistakes

- Using dense adjacency for huge graphs
- Random edge/node split causing leakage
- Too many GNN layers causing oversmoothing
- Ignoring edge features when important
- Treating graph order as meaningful

13. Edge Cases / Limitations

GNNs can oversmooth after many layers. They may struggle with long-range graph dependencies and dynamic graphs. Large-scale graphs require sampling.

14. Variations

Variation	What changes	When to use	Importance
GCN	Normalized neighbor averaging	Basic graph learning	High
GraphSAGE	Samples neighbors	Large graphs	High
GAT	Attention over neighbors	Important neighbors differ	High
GIN	Strong graph isomorphism power	Graph classification	Medium
R-GCN	Relation-specific edges	Knowledge graphs	Medium

15. Related Topics

GNN vs CNN: both aggregate local neighborhoods; CNN neighborhoods are fixed grids, GNN neighborhoods are arbitrary. GAT uses attention. Knowledge graphs connect GNNs to retrieval and reasoning.

16. Interview Questions

1. What is a GNN?
A neural network for graph-structured data.
2. What is message passing?
Nodes aggregate information from neighbors.
3. Why aggregation must be permutation-invariant?
Neighbor order has no inherent meaning.
4. What is oversmoothing?
Node embeddings become too similar after many layers.
5. Node vs graph classification?
Node predicts labels per node; graph predicts one label per graph.
6. What is GraphSAGE?
A sampling-based GNN for large graphs.
7. What is GAT?
A GNN using attention over neighbors.
8. Why add self-loops?
To include a node's own features in updates.
9. What is adjacency matrix?
Matrix representing graph edges.
10. How avoid leakage in graph ML?
Split carefully by time, graph, node, or edge depending on task.

17. Practice Tasks

- Implement GCN on a toy graph.
- Train node classifier on Cora.
- Compare GCN and GAT.
- Detect fraud communities.
- Analyze oversmoothing by increasing layers.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Molecule Toxicity GNN	Predicts chemical toxicity	PyTorch Geometric	Tox21	Research-oriented
Fraud Graph Detector	Finds suspicious accounts	PyG, NetworkX	Elliptic	Fintech AI
Citation Classifier	Classifies papers	PyG	Cora/PubMed	GNN basics

19. Quick Revision

- Key idea: learn by aggregating neighbors.
- Main formula: $H' = \sigma(D^{-1/2} A D^{-1/2} H W)$.
- When to use: relational graph data.
- Important metrics: F1, ROC-AUC, MRR.
- Common traps: leakage and oversmoothing.
- Interview one-liner: GNNs generalize neural networks to arbitrary graph neighborhoods.

20. Final Cheat Sheet

Item	Answer
Definition	Neural message passing on graphs
Input/output	Nodes/edges/features to node/edge/graph predictions
Main steps	Aggregate, update, readout
Key hyperparameters	Layers, hidden dim, aggregator, sampling
Metrics	Accuracy, F1, ROC-AUC, MRR
Pros	Uses relational structure
Cons	Oversmoothing, scaling difficulty
Best use cases	Molecules, recommender systems, fraud graphs

Neural ODE

1. Overview

Neural Ordinary Differential Equations model hidden state transformations as continuous-time dynamics instead of discrete layers. A neural network parameterizes the derivative of the hidden state.

They are used in continuous-time modeling, irregular time series, physics-informed ML, generative flows, and scientific machine learning.

2. Intuition

A ResNet updates state in steps: $h_{t+1} = h_t + f(h_t)$. Neural ODE takes infinitely small steps and describes how the state continuously changes over time.

3. Prerequisites

- Differential equations
- ResNet intuition
- Numerical solvers
- Backpropagation
- Continuous-time systems

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
ODE function	Neural net for derivative	Defines dynamics	velocity field	$dh/dt=f(h,t)$

Subtopic	Meaning	Why it matters	Example	Interview angle
Solver	Numerically integrates ODE	Computes final state	Euler/RK4	Accuracy vs speed
Continuous depth	Layers become time	Flexible computation	from t_0 to t_1	ResNet connection
Adjoint method	Memory-efficient gradients	Saves memory	backward ODE	Tradeoffs
Irregular time	Natural time intervals	Handles uneven samples	medical visits	Why Neural ODE

5. Algorithm / Working Process

Encode input into initial hidden state. Define derivative function with a neural network. Use ODE solver to integrate from start time to end time. Use final state for prediction. During training, gradients are computed through the solver or adjoint method.

6. Mathematical Foundation

Continuous dynamics:

$$\begin{aligned} dh(t)/dt &= f_{\theta}(h(t), t) \\ h(t_1) &= h(t_0) + \int_{t_0}^{t_1} f_{\theta}(h(t), t) dt \end{aligned}$$

ResNet relation:

$$h_{k+1} = h_k + f(h_k)$$

This resembles Euler discretization of an ODE.

7. Practical Implementation

```
import torch
from torch import nn

class ODEFunc(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(dim, 64), nn.Tanh(), nn.Linear(64, dim))

    def forward(self, h):
        return self.net(h)
```

```
def euler_solve(func, h, steps=10, dt=0.1):
    for _ in range(steps):
        h = h + dt * func(h)
    return h

class TinyNeuralODE(nn.Module):
    def __init__(self, dim=2, classes=2):
        super().__init__()
        self.func = ODEFunc(dim)
        self.head = nn.Linear(dim, classes)

    def forward(self, x):
        h = euler_solve(self.func, x)
        return self.head(h)
```

8. Code Explanation

`ODEFunc` predicts the derivative of hidden state. `euler_solve` performs a simple numerical integration. Production implementations usually use `torchdiffeq` solvers instead of this minimal Euler loop.

9. Training / Evaluation

Prepare data depending on task: continuous trajectories, irregular observations, or classification inputs. Metrics include trajectory MSE, classification accuracy, negative log likelihood, and physical consistency. Tune solver tolerance, integration time, hidden dimension, and network size.

10. Complexity and Cost

Cost depends on number of function evaluations by the solver. Adaptive solvers can become slow if dynamics are stiff. Memory can be reduced using adjoint methods but may introduce numerical issues.

11. Common Use Cases

- Irregular medical time series
- Physical system modeling
- Continuous normalizing flows
- Scientific ML
- Trajectory forecasting

12. Common Mistakes

- Treating solver as free computation
- Ignoring stiffness
- Using Neural ODE when a ResNet is enough
- Not checking numerical stability
- Overlooking solver tolerance impact

13. Edge Cases / Limitations

Neural ODEs can be slow, hard to train, and sensitive to solver choices. They are not automatically better than ResNets for standard supervised learning.

14. Variations

Variation	What changes	When to use	Importance
Augmented Neural ODE	Adds latent dimensions	More expressive flows	Medium
Latent ODE	ODE in latent space	Irregular time series	High
Neural CDE	Controlled differential equation	Continuous streams	Research
Continuous Normalizing Flow	ODE for density transform	Generative modeling	Medium

15. Related Topics

Neural ODE connects to ResNet through continuous-depth interpretation. It relates to physics-informed neural networks and state space models, but Neural ODE uses numerical integration explicitly.

16. Interview Questions

1. What is a Neural ODE?
A model where a neural net parameterizes continuous hidden dynamics.
2. How is it related to ResNet?
ResNet resembles Euler discretization of an ODE.
3. What does the ODE solver do?
Integrates derivative function over time.
4. What is adjoint method?
A memory-efficient way to compute gradients through ODE solve.
5. When use Neural ODE?
Continuous-time or irregularly sampled data.
6. What is a downside?
Solver cost and numerical instability.
7. What is stiffness?
Dynamics requiring very small solver steps.
8. Are Neural ODEs always better than ResNets?
No, often ResNets are simpler and faster.
9. What is latent ODE?
ODE dynamics in latent space.
10. What controls accuracy/speed?
Solver type and tolerance.

17. Practice Tasks

- Fit a spiral trajectory.
- Compare Euler and RK4 solvers.
- Train classifier with ODE block.
- Study effect of solver step count.
- Model irregular time-series observations.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Irregular Vitals Predictor	Models patient vitals	PyTorch	MIMIC-style data	Research depth
Physics Trajectory Model	Learns motion dynamics	PyTorch	synthetic pendulum	Scientific ML
Continuous Flow Toy Model	Learns density transform	PyTorch	2D moons	Generative modeling

19. Quick Revision

- Key idea: continuous-depth neural dynamics.
- Main formula: $\frac{dh}{dt} = f_{\text{theta}}(h, t)$.
- When to use: irregular/continuous-time systems.
- Important metrics: MSE, NLL, accuracy.
- Common traps: solver cost, using it unnecessarily.
- Interview one-liner: Neural ODE replaces discrete layers with learned continuous-time evolution.

20. Final Cheat Sheet

Item	Answer
Definition	Neural network-defined ODE dynamics
Input/output	Initial state to final state/prediction
Main steps	Define derivative, integrate, predict
Key hyperparameters	Solver, tolerance, time interval, hidden dim
Metrics	MSE, NLL, accuracy
Pros	Natural for continuous time
Cons	Solver cost and instability
Best use cases	Scientific and irregular time-series ML

Capsule Network

1. Overview

Capsule Networks represent features as vectors or matrices instead of scalar activations. A capsule encodes both presence and pose information such as position, orientation, scale, or deformation.

They were proposed to improve viewpoint understanding and part-whole relationships in vision, though they are less common in production than CNNs and Transformers.

2. Intuition

A CNN may detect eyes, nose, and mouth but can be fooled if they are arranged incorrectly. Capsule Networks try to model whether parts agree on the existence and pose of a whole object.

3. Prerequisites

- CNNs
- Vector norms
- Routing algorithms
- Matrix transformations
- Classification loss

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Capsule	Vector feature unit	Encodes pose and probability	digit part	Scalar neuron vs capsule
Squashing	Keeps vector length in range	Length means probability	norm near 1	Formula
Dynamic routing	Agreement-based connection	Learns part-whole relation	eyes vote for face	Routing by agreement
Pose	Object properties	Better viewpoint handling	rotated digit	Equivariance
Margin loss	Class capsule objective	Encourages correct capsule length	digit class vector	Loss function

5. Algorithm / Working Process

Lower-level capsules produce vectors. They are transformed into predictions for higher-level capsules. Routing iteratively increases coupling between lower and higher capsules that agree. Final class capsule lengths represent class probabilities.

6. Mathematical Foundation

Prediction vector:

$$\mathbf{u}_{\hat{j|i}} = \mathbf{W}_{ij} \mathbf{u}_i$$

Squash function:

$$\mathbf{v}_j = (||\mathbf{s}_j||^2 / (1 + ||\mathbf{s}_j||^2)) * (\mathbf{s}_j / ||\mathbf{s}_j||)$$

Margin loss:

$$L_k = T_k \max(0, m+ - ||v_k||)^2 + \lambda(1-T_k) \max(0, ||v_k|| - m-)^2$$

7. Practical Implementation

```
import torch
from torch import nn

def squash(s, eps=1e-8):
    norm = torch.norm(s, dim=-1, keepdim=True)
    scale = norm.pow(2) / (1 + norm.pow(2))
    return scale * s / (norm + eps)

class CapsuleLength(nn.Module):
    def forward(self, capsules):
        return torch.norm(capsules, dim=-1)

caps = torch.randn(8, 10, 16)
probs = CapsuleLength()(squash(caps))
print(probs.shape)
```

8. Code Explanation

The squash function preserves vector direction while limiting vector length. Capsule class probability is represented by vector norm, not by a scalar logit.

9. Training / Evaluation

Capsule Networks are usually trained on image classification datasets such as MNIST or smallNORB. Evaluate accuracy and robustness to viewpoint changes. Training is sensitive to routing iterations and implementation details.

10. Complexity and Cost

Dynamic routing is expensive compared with normal convolutions. The transformation matrices between capsules increase parameters and memory.

11. Common Use Cases

- Viewpoint-aware image recognition research
- Small object recognition tasks
- Part-whole relationship modeling
- Robustness experiments

12. Common Mistakes

- Treating capsule output as normal scalar activation
- Ignoring routing cost
- Expecting capsules to replace CNNs in production
- Misimplementing squash normalization
- Comparing only on MNIST

13. Edge Cases / Limitations

Capsules are computationally heavy and have not scaled as successfully as CNNs/Transformers. Routing can be unstable and slow.

14. Variations

Variation	What changes	When to use	Importance
Dynamic routing CapsNet	Agreement routing	Classic capsules	Medium
Matrix capsules	Matrix pose representation	More explicit pose	Research
EM routing	Expectation-maximization routing	Complex capsules	Research

15. Related Topics

Capsule Network vs CNN: capsules encode pose vectors and part-whole agreement; CNNs use scalar feature maps. Capsule equivariance differs from CNN translation equivariance.

16. Interview Questions

1. What is a capsule?
A vector/matrix feature unit encoding presence and pose.
2. Why vector length?
It represents probability of entity presence.
3. What is dynamic routing?
Iteratively assigning lower capsules to agreeing higher capsules.
4. What problem do capsules target?
Better part-whole and viewpoint modeling.
5. What is squash function?
Non-linearity that bounds capsule vector length.
6. Capsule vs neuron?
Capsule outputs vector; neuron outputs scalar.
7. Why are capsules not widely used?
Routing is expensive and scaling is difficult.
8. What is margin loss?
Loss encouraging correct class capsule to have high norm.
9. What is pose information?
Position, orientation, scale, deformation.
10. Are capsules important for placements?
Know conceptually; less common in coding rounds.

17. Practice Tasks

- Implement squash function.
- Train simple CapsNet on MNIST.
- Compare rotated digit performance with CNN.
- Visualize capsule lengths.
- Experiment with routing iterations.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Rotated Digit CapsNet	Tests viewpoint robustness	PyTorch	MNIST/rotated MNIST	Research curiosity
Part-Whole Demo	Visualizes routing	PyTorch, Streamlit	Synthetic shapes	Explains advanced DL
Capsule vs CNN Study	Benchmarks models	PyTorch	smallNORB	Research internship value

19. Quick Revision

- Key idea: vector capsules model part-whole pose relationships.
- Main formula: squash vector norm.
- When to use: research on viewpoint/part-whole modeling.
- Important metrics: accuracy, robustness.
- Common traps: routing cost.
- Interview one-liner: Capsule Networks replace scalar features with pose-aware vector capsules.

20. Final Cheat Sheet

Item	Answer
Definition	Pose-aware vector neural units
Input/output	Image features to class capsule lengths
Main steps	Capsules, transform votes, route, classify
Key hyperparameters	Capsule dim, routing iterations, margins
Metrics	Accuracy, robustness
Pros	Models part-whole relationships
Cons	Expensive and less scalable
Best use cases	Research demos and viewpoint tasks

Mamba / State Space Models

1. Overview

State Space Models process sequences using learned recurrent dynamics. Mamba is a modern selective state space architecture designed to handle long sequences efficiently, competing with Transformers in some language and sequence modeling settings.

SSMs are used in long-context modeling, audio, genomics, time series, and efficient sequence modeling.

2. Intuition

An SSM keeps a compact hidden state that evolves as it reads a sequence. Mamba improves this by making the state update selective: the model decides which information to keep or ignore based on the current token.

3. Prerequisites

- Sequence modeling
- Linear dynamical systems
- RNN intuition
- Convolution and recurrence
- Transformer limitations

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
State	Compact memory	Carries past information	summary of previous tokens	SSM vs attention
State transition	Updates memory	Defines dynamics	$h_t = Ah_{t-1} + Bx_t$	Linear system
Selectivity	Input-dependent parameters	Keeps relevant tokens	remember keyword	Why Mamba
Linear-time scaling	Cost grows linearly with length	Long context efficiency	million-token signal	Transformer comparison
Scan operation	Efficient recurrence computation	Parallel-friendly implementation	prefix computation	Hardware-aware design

5. Algorithm / Working Process

Traditional SSM updates hidden state using learned transition, input, and output matrices. Mamba makes parts of this update input-dependent, allowing content-based selection. It processes sequences in linear time and uses hardware-aware parallel scan kernels.

6. Mathematical Foundation

Basic continuous SSM:

$$\begin{aligned}h'(t) &= A h(t) + B x(t) \\ y(t) &= C h(t) + D x(t)\end{aligned}$$

Discrete form:

$$\begin{aligned}h_t &= A_{\text{bar}} h_{t-1} + B_{\text{bar}} x_t \\ y_t &= C h_t + D x_t\end{aligned}$$

Mamba-style selectivity makes parameters such as B , C , or step size depend on input x_t .

7. Practical Implementation

```
import torch
from torch import nn

class MinimalSelectiveSSM(nn.Module):
    def __init__(self, dim, state_dim=32):
        super().__init__()
        self.to_state = nn.Linear(dim, state_dim)
        self.gate = nn.Linear(dim, state_dim)
        self.out = nn.Linear(state_dim, dim)

    def forward(self, x):
        h = torch.zeros(x.size(0), self.to_state.out_features, device=x.device)
        ys = []
        for t in range(x.size(1)):
            candidate = torch.tanh(self.to_state(x[:, t]))
            keep = torch.sigmoid(self.gate(x[:, t]))
            h = keep * h + (1 - keep) * candidate
            ys.append(self.out(h))
        return torch.stack(ys, dim=1)
```

8. Code Explanation

This is not full Mamba; it is a small selective-state teaching model. The gate controls how much previous state is kept. Full Mamba uses specialized parameterization and optimized scan kernels.

9. Training / Evaluation

Train like sequence models using next-token loss, classification loss, or forecasting losses. Evaluate perplexity, accuracy, F1, MAE/RMSE, throughput, latency, and memory use over long contexts.

10. Complexity and Cost

SSMs can scale linearly with sequence length, unlike standard Transformer attention's quadratic cost. Full performance depends heavily on optimized kernels.

11. Common Use Cases

- Long sequence modeling
- Genomics
- Audio modeling
- Time-series forecasting
- Efficient language modeling

12. Common Mistakes

- Calling the toy recurrence "full Mamba"
- Ignoring kernel implementation details
- Assuming SSMs always beat Transformers
- Forgetting that content-based retrieval is different from attention
- Benchmarking only short sequences

13. Edge Cases / Limitations

SSMs may be weaker than attention on tasks requiring exact retrieval of arbitrary previous tokens. Ecosystem support is smaller than Transformer tooling.

14. Variations

Variation	What changes	When to use	Importance
S4	Structured state space model	Long signals	Medium
DSS	Diagonal SSM	Simpler SSM	Research
Mamba	Selective SSM	Long efficient sequences	High
Hybrid SSM-Attention	Combines both	Retrieval plus efficiency	High

15. Related Topics

Mamba vs Transformer: Mamba uses selective state updates with linear scaling; Transformer uses attention with direct pairwise interactions. Mamba relates to RNNs but is designed for efficient long-sequence training.

16. Interview Questions

1. What is a state space model?
A model with hidden state evolving over time.
2. What is Mamba?
A selective state space sequence model.
3. Why is Mamba efficient for long sequences?
It scales roughly linearly with sequence length.
4. What is selectivity?
Input-dependent control over what information enters or leaves state.
5. SSM vs RNN?
Both maintain state; modern SSMs use structured dynamics and efficient scans.

6. SSM vs Transformer?
SSM compresses past into state; Transformer attends directly to tokens.
7. What is the limitation of compressed state?
It may lose exact details needed for retrieval.
8. Where are SSMs strong?
Long signals, audio, genomics, time series.
9. Why hardware-aware implementation?
Efficient scan kernels are key to speed.
10. Are SSMs replacing Transformers?
Not universally; hybrids are common research direction.

17. Practice Tasks

- Implement simple state update.
- Compare RNN and SSM-like model on long copy task.
- Benchmark sequence length vs runtime.
- Train on time-series forecasting.
- Read Mamba paper and summarize selectivity.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Long Signal Classifier	Classifies long sensor streams	PyTorch	UCR archive	Efficient sequence ML
Genomic Sequence Model	Predicts DNA labels	PyTorch	Genomic benchmarks	Research value
Transformer vs SSM Benchmark	Compares scaling	PyTorch	synthetic tasks	Strong interview discussion

19. Quick Revision

- Key idea: selective hidden state for long sequences.
- Main formula: $h_t = A h_{t-1} + B x_t$.
- When to use: long sequences where attention is costly.
- Important metrics: perplexity, accuracy, throughput, memory.
- Common traps: overclaiming vs Transformers.
- Interview one-liner: Mamba is a selective SSM that trades quadratic attention for efficient state-based sequence modeling.

20. Final Cheat Sheet

Item	Answer
Definition	Selective state space sequence model
Input/output	Sequence to sequence/features

Item	Answer
Main steps	Update state, select information, emit output
Key hyperparameters	State dim, model dim, layers, sequence length
Metrics	Perplexity, F1, throughput
Pros	Long-sequence efficiency
Cons	Less direct retrieval than attention
Best use cases	Long signals, audio, genomics, efficient LM

Mixture of Experts

1. Overview

Mixture of Experts is an architecture where different expert subnetworks specialize in different inputs, and a router chooses which experts process each example or token. Modern sparse MoE Transformers increase model capacity without activating all parameters for every token.

MoE is used in large language models, recommendation systems, multitask learning, and scalable AI systems.

2. Intuition

Instead of one general doctor handling every case, a router sends heart cases to a cardiologist and skin cases to a dermatologist. MoE lets parts of a model specialize.

3. Prerequisites

- Neural networks
- Softmax routing
- Transformers
- Load balancing
- Distributed training basics

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Expert	Specialized subnetwork	Adds capacity	FFN expert in Transformer	Expert specialization
Router/gate	Selects experts	Controls computation	top-2 routing	Routing loss
Sparse activation	Only few experts active	Efficient large models	2 of 64 experts	Params vs FLOPs

Subtopic	Meaning	Why it matters	Example	Interview angle
Load balancing	Avoids expert collapse	Keeps training stable	equal token distribution	Auxiliary loss
Capacity factor	Expert token limit	Controls overflow	max tokens per expert	Dropped tokens

5. Algorithm / Working Process

For each input token, router computes scores over experts. Select top-k experts. Send token representation to selected experts. Combine expert outputs using routing weights. Add auxiliary losses to balance expert usage.

6. Mathematical Foundation

Router probabilities:

$$p(e \mid x) = \text{softmax}(W_r \cdot x)$$

Top-k MoE output:

$$y = \sum_{e \in \text{TopK}(p)} p(e|x) * \text{Expert}_e(x)$$

Training loss:

$$L_{\text{total}} = L_{\text{task}} + \alpha * L_{\text{load_balance}}$$

7. Practical Implementation

```
import torch
from torch import nn
import torch.nn.functional as F

class TinyMoE(nn.Module):
    def __init__(self, dim=64, experts=4):
        super().__init__()
        self.router = nn.Linear(dim, experts)
        self.experts = nn.ModuleList([
            nn.Sequential(nn.Linear(dim, 4 * dim), nn.ReLU(), nn.Linear(4 * dim,
dim))
            for _ in range(experts)
        ])

    def forward(self, x):
        weights = F.softmax(self.router(x), dim=-1)
```

```
expert_outputs = torch.stack([expert(x) for expert in self.experts],
                              dim=-2)
return (weights.unsqueeze(-1) * expert_outputs).sum(dim=-2)

model = TinyMoE()
x = torch.randn(8, 10, 64)
print(model(x).shape)
```

8. Code Explanation

This dense teaching version runs all experts and combines them by router weights. Production sparse MoE runs only top-k experts to save compute. The router maps each token to expert probabilities.

9. Training / Evaluation

Train with the task loss plus routing/load-balancing losses. Monitor expert usage, dropped tokens, throughput, memory, and validation performance. In LLMs, evaluate perplexity, downstream benchmarks, latency, and serving cost.

10. Complexity and Cost

Sparse MoE increases total parameters while keeping active parameters per token limited. Distributed training is complex because tokens must be routed across devices.

11. Common Use Cases

- Large language models
- Multilingual models
- Recommendation systems
- Multi-domain prediction
- Multitask learning

12. Common Mistakes

- Running all experts and expecting sparse speedup
- Ignoring load balancing
- Letting router collapse to few experts
- Not accounting for communication overhead
- Comparing parameter count without active FLOPs

13. Edge Cases / Limitations

MoE is hard to train and serve efficiently. Routing can be unstable. Expert parallelism increases communication complexity.

14. Variations

Variation	What changes	When to use	Importance
Dense MoE	Uses all experts	Small models/research	Medium
Sparse top-k MoE	Uses selected experts	Large scalable models	High
Switch Transformer	Top-1 expert routing	Simpler sparse MoE	High
Hierarchical MoE	Multi-level routing	Many experts	Research

15. Related Topics

MoE vs ensemble: MoE routes within one model; ensembles combine separate models. MoE Transformer replaces or augments FFN blocks with experts. MoE vs LoRA: MoE changes architecture; LoRA adapts model weights efficiently.

16. Interview Questions

1. What is MoE?
A model with multiple experts selected by a router.
2. Why use MoE?
To increase capacity without activating all parameters.
3. What is sparse MoE?
Only top-k experts process each token.
4. What is router collapse?
Most tokens go to a few experts.
5. How prevent collapse?
Load-balancing auxiliary loss and capacity controls.
6. What is capacity factor?
Limit on tokens each expert can process.
7. MoE vs ensemble?
MoE is routed internally; ensemble combines independent model outputs.
8. Why is MoE hard to serve?
Routing and cross-device communication.
9. What is Switch Transformer?
Top-1 routed sparse MoE Transformer.
10. Does more total parameters mean more inference cost?
Not necessarily; active parameters matter.

17. Practice Tasks

- Implement dense MoE layer.
- Modify it to top-1 routing.
- Track expert usage histogram.
- Add load-balancing loss.
- Compare MoE and single FFN on multi-domain data.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Domain-Routed Classifier	Routes examples to experts	PyTorch	Multi-domain reviews	Advanced modeling
Toy MoE Transformer	Replaces FFN with experts	PyTorch	tiny text corpus	LLM architecture depth
Expert Usage Dashboard	Visualizes routing patterns	PyTorch, Streamlit	synthetic/classes	Interpretability demo

19. Quick Revision

- Key idea: route inputs to specialized experts.
- Main formula: $y = \sum p(e|x) \text{Expert}_e(x)$.
- When to use: scaling capacity or multi-domain specialization.
- Important metrics: task metric, expert balance, latency.
- Common traps: router collapse, communication cost.
- Interview one-liner: MoE increases model capacity by activating only selected expert networks per token.

20. Final Cheat Sheet

Item	Answer
Definition	Router-selected expert architecture
Input/output	Token/example to expert-combined output
Main steps	Route, expert compute, combine, balance
Key hyperparameters	Expert count, top-k, capacity factor, aux loss
Metrics	Accuracy/perplexity, expert load, latency
Pros	Huge capacity with sparse compute
Cons	Complex training and serving
Best use cases	Large-scale LLMs and multi-domain systems